Backend Project Java

Professional Study Guide

FakeStore API Implementation with Spring Boot

Scaler Academy Backend Project Module

Project: productService | Java 17 | Spring Boot 4.1

Table of Contents

Chapter 1: Module Overview & Intro to Backend

Theory: Beginner to Advanced

Beginner: What Is Backend Development?

Backend development is the server-side layer of an application. It receives HTTP requests from clients (browsers, mobile apps), applies business logic, talks to databases or external APIs, and returns structured responses (usually JSON). In this module you build productService: a Spring Boot 4.1 application on Java 17 that proxies FakeStore API (fakestoreapi.com) behind your own REST endpoints.

Intermediate: Client-Server Architecture

A typical backend follows a layered pattern: Controller (HTTP boundary) -> Service (business logic) -> Repository or External Client (data access). productService skips a database and uses RestClient to call FakeStore instead. Controllers live in org.pavan.productservice.controllers; services in services; DTOs in dtos; domain models in models; configuration in config; exceptions in exceptions.

Advanced: API Gateway / BFF Pattern

productService acts as a Backend-for-Frontend (BFF) or thin API gateway. It translates FakeStore JSON into your own domain models (Product, Category), adds centralized error handling via GlobalExceptionHandler, and exposes a consistent URL namespace (/products, /categories, /carts, /users, /auth). In production, this layer would add auth, rate limiting, caching, and schema validation before hitting upstream services.

Why Do We Need Backend Development?

Clients cannot safely store API keys, business rules, or direct database credentials. Backend code enforces validation, hides third-party URLs (fakestore.api.base-url in application.properties), and provides a stable contract even when upstream APIs change. Without a backend, every frontend would duplicate RestClient calls, error mapping, and DTO conversion.

Module Goal

Build a production-shaped Spring Boot service that integrates with a real external API, implements full CRUD for products/carts/users, and demonstrates professional patterns: DI, layered architecture, global exception handling, and REST conventions.

Real-World Analogy

Think of FakeStore API as a wholesale warehouse. Your Spring Boot app is the retail store front desk. Customers (frontends) never enter the warehouse; they ask the clerk (ProductController) for items. The clerk checks the internal catalog system (ProductService + RestClient), translates warehouse labels (ProductDto) into customer-friendly tags (Product model), and handles 'item not found' politely (ProductNotFoundException -> 404 JSON).

Internal Working

```
Client (Postman / React)
         |
         |  HTTP GET /products/5
         v
+--------------------+
```

```
| ProductController   |  @RestController @RequestMapping("/products")
+----------+----------+
           |
           v
+--------------------+
| FakeStoreProduct    |  @Service implements ProductService
| Implementation      |
+----------+----------+
           |
           |  RestClient GET /products/5
           v
+--------------------+
| fakestoreapi.com    |  External JSON API
+--------------------+
           |
           |  ProductDto JSON
           v
+--------------------+
| ProductMapper       |  DTO -> Product domain model
+--------------------+
           |
           v
     ResponseEntity<Product>  -->  Client
```

Practical Example: productService Bootstrap

Entry point: ProductServiceApplication.java with @SpringBootApplication. RestClientConfig creates a RestClient bean with base URL from properties. ProductController constructor-injects ProductService; Spring resolves FakeStoreProductImplementation automatically.

```
// ProductServiceApplication.java
@SpringBootApplication
public class ProductServiceApplication {
    public static void main(String[] args) {
        SpringApplication.run(ProductServiceApplication.class, args);
    }
}

// application.properties
spring.application.name=productService
fakestore.api.base-url=https://fakestoreapi.com
```

Issues in Current Code

Main2.java is unrelated scratch code left in the main package; it should be deleted. There is no README or OpenAPI spec documenting endpoints. The test class ProductServiceApplicationTests only checks context loads; no controller or service tests exist. DevTools is included but hot-reload behavior is undocumented.

Improved Version: Current -> Better -> Production

Aspect	Current	Better	Production
Structure	Flat packages	Add README + API docs	OpenAPI/Swagger UI
Config	Hardcoded in props	Profile-based URLs	Vault + env vars
Health	Actuator dep only	Expose /actuator/health	K8s probes + metrics
Tests	Empty context test	MockMvc controller tests	Contract tests vs FakeStore
Cleanup	Main2.java present	Remove dead code	CI lint + arch tests

Current State

Minimal Spring Boot app with five controllers, five FakeStore service implementations, one RestClient bean, and a GlobalExceptionHandler. Functional for learning but not deployment-ready.

Better State

Add springdoc-openapi, @Valid on request bodies, integration tests with MockRestServiceServer, remove Main2.java, return DTOs consistently from all controllers (ProductController currently returns Product entity while CategoryController returns ProductDto).

Production State

Deploy behind API gateway with JWT auth, circuit breaker (Resilience4j) on RestClient, distributed tracing (Micrometer), secrets management, and horizontal scaling with stateless pods.

Revision Table

Term	Definition	In productService
Backend	Server-side app logic	Spring Boot productService
REST	HTTP resource API	Controllers at /products etc.
DTO	Data transfer object	ProductDto, CartDto, UserDto
Domain Model	Internal representation	Product, Category
BFF	Backend for frontend	Proxies FakeStore API
Dependency Injection	Spring provides beans	Constructor injection
External API	Third-party HTTP service	fakestoreapi.com

Interview Questions & Answers

Q: What is the difference between frontend and backend?
A: Frontend runs in the browser and handles UI/UX. Backend runs on a server, processes requests, enforces business rules, and accesses data sources. productService is purely backend; it has no HTML or UI.

Q: What technologies does this project use?
A: Java 17, Spring Boot 4.1, spring-boot-starter-webmvc, Lombok, RestClient (built into Spring 6.1+), and Actuator. No database; data comes from FakeStore API.

Q: What is the role of application.properties?
A: Externalized configuration. fakestore.api.base-url lets you change the upstream API URL without recompiling. RestClientConfig reads it via @Value injection.

Q: Why proxy FakeStore instead of calling it from the frontend?
A: Security (hide upstream details), consistency (your own error format via ErrorResponseDto), transformation (ProductMapper), and future flexibility (swap FakeStore for a real database without changing clients).

Q: What packages exist and what is each responsible for?
A: controllers: HTTP layer. services: business logic + RestClient calls. dtos: JSON shapes. models: domain objects. config: beans. exceptions: error handling. mappers: DTO-model conversion.

Q: What is a RESTful resource in this project?
A: Products (/products), categories (/categories), carts (/carts), users (/users), and auth (/auth/login). Each maps to FakeStore endpoints.

Q: How does Spring Boot simplify backend development?

A: Auto-configuration, embedded Tomcat, component scanning, and starter dependencies reduce boilerplate. @SpringBootApplication enables all of this with one annotation.

Q: What is the default server port?

A: 8080 unless changed in application.properties. All controllers are served from http://localhost:8080.

Q: What problem does layered architecture solve?

A: Separation of concerns. Controllers should not contain HTTP client code; services should not parse HTTP headers. Changes to FakeStore URLs only affect service layer.

Q: What is wrong with having Main2.java in the project?

A: It is dead code unrelated to the application. It confuses reviewers, may run accidentally, and violates clean project structure. Delete it.

Chapter 2: Version Control & Git

Theory: Beginner to Advanced

Beginner: What Is Git?

Git is a distributed version control system. It tracks every change to source code as commits (snapshots), lets multiple developers work in parallel via branches, and merges changes back together. For productService, Git would track controllers, services, DTOs, and configuration files independently.

Intermediate: Commits, Branches, Remotes

A commit is an immutable snapshot with a message and SHA hash. A branch is a movable pointer to a commit (e.g., feature/cart-api). A remote (origin) is a copy on GitHub. git push uploads local commits; git pull fetches and merges remote changes. Typical workflow for adding CartController: branch -> implement -> commit -> push -> PR.

Advanced: Merge vs Rebase, Cherry-pick, Stash

Merge creates a merge commit combining two branch histories. Rebase replays your commits on top of another branch, producing linear history. Cherry-pick applies a single commit from one branch to another. Stash temporarily shelves uncommitted work. Fork creates your own copy of a repo on GitHub; PR proposes merging your fork's branch into upstream.

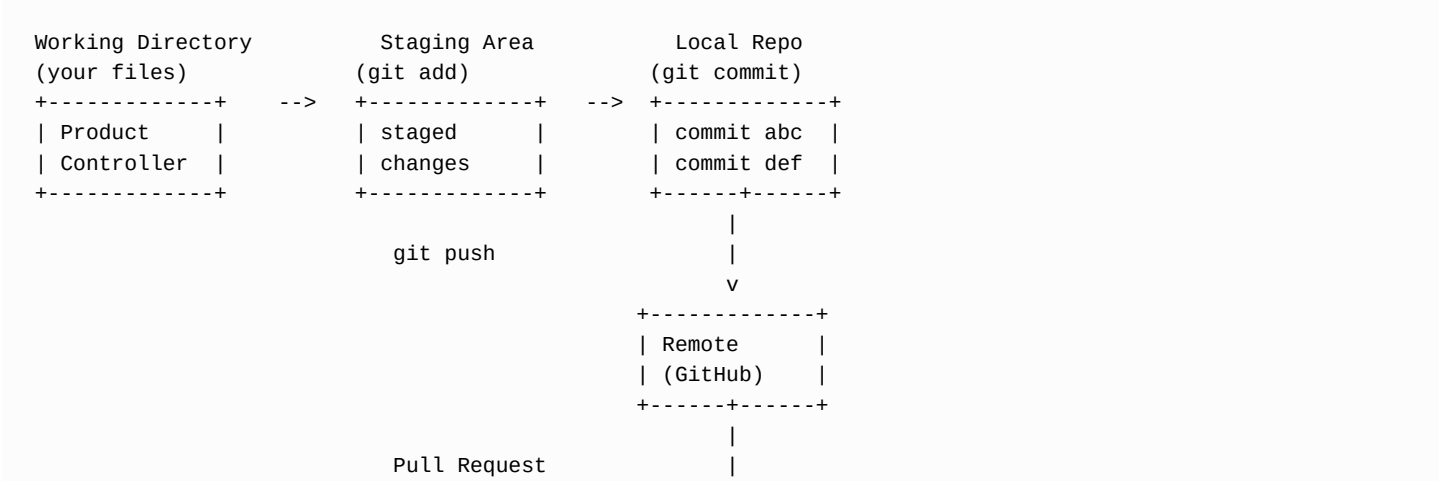
Why Do We Need Git?

Without version control, fixing the category endpoint (from ID to category name) would risk losing the old code with no rollback. Git provides history, blame, code review via PRs, and collaboration. When FakeStoreProductImplementation gained PUT/DELETE methods, Git would show exactly when and why each method was added.

Real-World Analogy

Git is like Google Docs version history for code. Commits are named saves. Branches are parallel drafts (one person writes CartController while another fixes CategoryController). Merge is combining drafts. Rebase is rewriting your draft pages to sit cleanly after a colleague's edits. Stash is putting your half-written paragraph in a drawer while you answer an urgent email.

Internal Working



```
                                          v
                                    +-------------+
                                    | main branch |
                                    | (reviewed)  |
                                    +-------------+

Rebase:  feature ---A---B---C
                  \
                   rebase onto main
Result:  main ---1---2---3---A'---B'---C'

Cherry-pick: pick commit B from feature -> apply on hotfix branch
```

Practical Example: productService Git Workflow

Suppose you implement missing cart endpoints in FakeStoreCartImplementation. A proper Git workflow:

```
git checkout -b feature/cart-crud
# Edit CartController.java, FakeStoreCartImplementation.java
git add src/main/java/org/pavan/productservice/controllers/CartController.java
git add src/main/java/org/pavan/productservice/services/FakeStoreCartImplementation.java
git commit -m "Add cart CRUD endpoints proxying FakeStore API"
git push -u origin feature/cart-crud
# Open PR on GitHub, request review, merge to main
```

Issues Relevant to This Project

If Main2.java was committed, it pollutes history. Dead files should never reach main. Large commits mixing cart + user + auth changes are hard to review. Each controller should be a separate commit or PR. No .gitignore issues visible, but target/ build output must stay untracked.

Improved Version: Current -> Better -> Production

Practice	Current Risk	Better	Production
Commits	Vague messages	Conventional commits	Linked to Jira tickets
Branches	Direct to main	Feature branches + PR	Protected main + CI
Reviews	None assumed	1 reviewer minimum	CODEOWNERS per package
History	Merge commits messy	Rebase feature branches	Squash merge on PR
Secrets	Props in repo	.env gitignored	Git-secrets scan in CI

Git Commands Cheat Sheet for This Module

```
git status                  # See modified files
git diff                    # Unstaged changes
git log --oneline -10       # Recent commits
git branch -a               # All branches
git stash push -m "wip"     # Save WIP
git stash pop               # Restore WIP
git rebase main             # Rebase current branch onto main
git cherry-pick <sha>       # Apply one commit elsewhere
git remote -v               # Show remotes
git fetch origin            # Download remote changes
git merge origin/main       # Merge remote main into current
```

Revision Table

Command	Purpose	When to Use

git commit	Save snapshot	After logical unit of work
git merge	Combine branches	Integrate feature to main
git rebase	Replay commits	Clean history before PR
git stash	Shelve changes	Switch branches with WIP
git cherry-pick	Copy one commit	Hotfix a single fix
git push	Upload commits	After local commits ready
git fork	Copy repo on GitHub	Contributing to others' repos
Pull Request	Propose merge	Code review gate

Interview Questions & Answers

Q: What is the difference between git merge and git rebase?
A: Merge preserves both histories and creates a merge commit. Rebase rewrites commits onto a new base for linear history. Never rebase shared/public branches.

Q: What is a merge conflict?
A: Occurs when two branches modify the same lines. Git cannot auto-merge; you must manually resolve, e.g., if two devs both edit ProductController.getAllProducts.

Q: What does git stash do?
A: Saves uncommitted changes temporarily so you can switch branches cleanly, then restore with git stash pop.

Q: What is cherry-pick used for?
A: Applying a specific commit from one branch to another without merging the entire branch. Useful for backporting a bug fix.

Q: What is the difference between fork and clone?
A: Clone copies a repo locally. Fork creates your own remote copy on GitHub. You clone your fork, push to it, then open PR to upstream.

Q: What is a Pull Request?
A: A request to merge one branch into another with code review, CI checks, and discussion before integration.

Q: What is origin?
A: Default name for the remote repository you cloned from or pushed to.

Q: What makes a good commit message?
A: Imperative mood, explains why not just what: 'Fix category endpoint to use name instead of ID' not 'fixed stuff'.

Q: What is git reset vs git revert?
A: reset moves branch pointer (can erase history locally). revert creates a new commit that undoes a previous one (safe for shared branches).

Q: How would you structure PRs for productService?
A: One PR per feature area: products CRUD, carts, users, auth, exception handling. Each PR should include tests and not mix unrelated changes.

Git Workflow for Category Endpoint Fix

When the category endpoint was changed from ID to name, a proper Git history would show:

```
git checkout -b fix/category-name-endpoint
# Edit CategoryController, ProductController, FakeStoreCategoryImplementation
git commit -m "fix: use category name instead of ID in path variables"
git commit -m "test: verify GET /products/category/electronics returns products"
git push -u origin fix/category-name-endpoint
# PR description: FakeStore API uses string category names not numeric IDs
```

Branching Strategy for Team Development

Recommended branch naming: feature/cart-crud, fix/password-leak, refactor/product-mapper. Main branch always deployable. Feature branches live 1-3 days max. Use git rebase main before opening PR to reduce merge conflicts.

Chapter 3: Spring Boot, DI & IoC

Theory: Beginner to Advanced

Beginner: Spring Boot Basics

Spring Boot is a framework that simplifies Java backend development. @SpringBootApplication on ProductServiceApplication enables auto-configuration, component scanning, and embedded Tomcat. You run the app with main() -- no external server deployment needed for development.

Intermediate: Dependency Injection (DI) and IoC

Inversion of Control (IoC): Spring creates and manages objects (beans) instead of you calling new. Dependency Injection: dependencies are provided via constructor, setter, or field. ProductController receives ProductService through constructor injection -- Spring finds FakeStoreProductImplementation (the @Service implementation) and injects it automatically.

Advanced: Bean Lifecycle, Scopes, Auto-config, Starters

Bean lifecycle: instantiate -> populate properties -> @PostConstruct -> use -> @PreDestroy -> destroy. Default scope is singleton (one RestClient bean for entire app). Other scopes: prototype (new each injection), request, session. spring-boot-starter-webmvc auto-configures DispatcherServlet, Jackson, Tomcat. RestClientConfig defines a custom @Bean. @Configuration classes are processed at startup by ConfigurationClassPostProcessor.

Why Do We Need Spring DI?

Without DI, ProductController would do: new FakeStoreProductImplementation(new RestClient(...)) -- tight coupling, untestable, hard to swap implementations. With DI, you program to ProductService interface; tests inject mocks; production injects FakeStore implementation.

Real-World Analogy

IoC container is a restaurant kitchen manager. You (the controller) order a dish (ProductService). The manager (Spring) decides which chef (FakeStoreProductImplementation) cooks it and which supplier (RestClient bean) provides ingredients. You never hire the chef directly.

Internal Working

```
@SpringBootApplication
ProductServiceApplication
         |
         v
+--------------------------+
| ApplicationContext (IoC)  |
|                          |
|  Beans:                  |
|  - restClient()          |  @Bean in RestClientConfig
|  - FakeStoreProductImpl  |  @Service
|  - ProductController     |  @RestController
|  - GlobalExceptionHandler |  @RestControllerAdvice
+--------------------------+
         |
         |  Constructor Injection
```

```
         v
ProductController(ProductService ps)
     ps = FakeStoreProductImplementation
     which needs RestClient (injected)

Bean Creation Order:
1. Scan @Configuration, @Service, @RestController
2. Create RestClient @Bean
3. Create FakeStoreProductImplementation(RestClient)
4. Create ProductController(ProductService)
```

Practical Example: productService DI Chain

```
// RestClientConfig.java - @Configuration provides RestClient bean
@Configuration
public class RestClientConfig {
    @Value("${fakestore.api.base-url}")
    private String baseUrl;

    @Bean
    public RestClient restClient() {
        return RestClient.builder().baseUrl(baseUrl).build();
    }
}

// FakeStoreProductImplementation - @Service registered as ProductService
@Service
public class FakeStoreProductImplementation implements ProductService {
    private final RestClient restClient;
    public FakeStoreProductImplementation(RestClient restClient) {
        this.restClient = restClient;  // DI
    }
}

// ProductController - @RestController receives ProductService
@RestController
public class ProductController {
    private final ProductService productService;
    public ProductController(ProductService productService) {
        this.productService = productService;  // DI
    }
}
```

Issues in Current Code

Only one ProductService implementation exists -- interface adds indirection without benefit until a second implementation (e.g., DatabaseProductService) is needed. ProductMapper is a static utility, not a Spring bean -- inconsistent with DI pattern. No @Profile to switch between FakeStore and local mock. RestClient has no customization (timeouts, interceptors). Lombok @Getter/@Setter on models but BaseModel has commented-out @Builder/@AllArgsConstructor.

Improved Version: Current -> Better -> Production

Aspect	Current	Better	Production
DI	Constructor only	Interface + mock for tests	@Qualifier for multi-impl
Mapper	Static ProductMapper	@Component mapper beans	MapStruct generated
Config	Single RestClient	RestClientCustomizer	Per-service clients

Scopes	All singleton	Prototype for stateful	Request scope where needed
Lifecycle	No @PostConstruct	Warm-up health check	Graceful shutdown hooks

Spring Boot Starters in pom.xml

spring-boot-starter-webmvc: MVC, Jackson, Tomcat. spring-boot-starter-actuator: health/metrics endpoints (not configured in application.properties). spring-boot-devtools: dev hot-reload. lombok: boilerplate reduction.

Bean Scopes Reference

Scope	Behavior	productService Usage
singleton	One per container	RestClient, all @Service beans
prototype	New per injection	Not used (would be for statefu
request	Per HTTP request	Not used (useful for user cont
session	Per HTTP session	Not used (would need spring-se

Revision Table

Annotation	Purpose	Example in Project
@SpringBootApplication	Enable auto-config + scan	ProductServiceApplication
@RestController	HTTP endpoint bean	ProductController
@Service	Business logic bean	FakeStoreProductImplementation
@Configuration	Bean factory class	RestClientConfig
@Bean	Register method return value	restClient()
@Value	Inject property	baseUrl in RestClientConfig
@RestControllerAdvice	Global controller advice	GlobalExceptionHandler

Interview Questions & Answers

Q: What is IoC?

A: Inversion of Control: framework controls object creation and lifecycle instead of application code calling new directly.

Q: Constructor vs field injection?

A: Constructor injection is recommended: immutable dependencies, testable, required deps explicit. Field injection hides dependencies and complicates testing.

Q: What does @SpringBootApplication combine?

A: @Configuration + @EnableAutoConfiguration + @ComponentScan.

Q: What is auto-configuration?

A: Spring Boot conditionally configures beans based on classpath. webmvc starter auto-configures DispatcherServlet and Jackson.

Q: Default bean scope?

A: Singleton: one instance shared across the application context.

Q: How does Spring know FakeStoreProductImplementation is a ProductService?

A: It implements ProductService interface and is annotated @Service. When ProductController needs ProductService, Spring injects the only implementation.

Q: What is @Value used for?

A: Injecting values from application.properties. RestClientConfig uses it for fakestore.api.base-url.

Q: What happens if two classes implement ProductService?

A: Spring throws NoUniqueBeanDefinitionException unless you use @Primary or @Qualifier.

Q: What is the bean lifecycle?

A: Instantiate, inject dependencies, @PostConstruct callbacks, ready, @PreDestroy on shutdown.

Q: Why is ProductMapper not a Spring bean?

A: It uses static methods only. Works but breaks DI consistency; MapStruct @Mapper(componentModel='spring') would be the idiomatic fix.

Chapter 4: REST APIs & HTTP/HTTPS

Theory: Beginner to Advanced

Beginner: What Is REST?

REST (Representational State Transfer) uses HTTP to operate on resources identified by URLs. productService exposes resources: /products, /categories, /carts, /users. Each resource supports appropriate HTTP methods. Responses use JSON and standard status codes.

Intermediate: HTTP Methods

GET: read (idempotent, safe). POST: create (non-idempotent). PUT: full update (idempotent). DELETE: remove (idempotent). PATCH: partial update (not used in this project). ProductController uses all four for /products. AuthController uses POST /auth/login only.

Advanced: Status Codes, Headers, HTTPS/TLS

2xx success (200 OK, 201 Created, 204 No Content). 4xx client error (400 Bad Request, 404 Not Found, 401 Unauthorized). 5xx server error (500 Internal Server Error, 502 Bad Gateway). Headers: Content-Type: application/json, Accept, Authorization (not implemented here). HTTPS/TLS encrypts HTTP traffic; fakestore.api.base-url uses HTTPS. Your local Spring app runs HTTP on 8080 unless SSL is configured.

Why Do We Need REST Conventions?

Predictable APIs let any client integrate without custom protocols. Returning 201 for POST (ProductController.addNewProduct), 204 for DELETE (deleteProduct), and 404 for missing resources (via GlobalExceptionHandler) follows HTTP semantics that tools, caches, and developers already understand.

Real-World Analogy

REST is a standardized postal system. GET is 'send me catalog page 5'. POST is 'register this new product'. PUT is 'replace product 5 entirely'. DELETE is 'remove product 5'. Status codes are delivery receipts: 200 delivered, 404 address not found, 500 sorting facility burned down.

Internal Working: HTTP Request/Response

```
Client Request:
+----------------------------------------+
| POST /products HTTP/1.1                |
| Host: localhost:8080                     |
| Content-Type: application/json           |
|                                        |
| {"title":"New Item","price":9.99,...}    |
+----------------------------------------+
                  |
                  v
         Spring DispatcherServlet
                  |
                  v
         ProductController.addNewProduct()
                  |
                  v
```

```
Server Response:
+----------------------------------------+
| HTTP/1.1 201 Created                   |
| Content-Type: application/json           |
|                                        |
| {"id":21,"title":"New Item",...}       |
+----------------------------------------+

HTTPS adds TLS handshake:
Client <--[encrypted]--> Server
Certificate validates fakestoreapi.com identity
```

Practical Example: productService Endpoints

Method	Endpoint	Status	Controller
GET	/products	200	ProductController
GET	/products/{id}	200/404	ProductController
POST	/products	201	ProductController
PUT	/products/{id}	200/404	ProductController
DELETE	/products/{id}	204/404	ProductController
GET	/categories	200	CategoryController
POST	/auth/login	200	AuthController
GET	/carts	200	CartController
GET	/users	200	UserController

Issues in Current Code

ProductController.getAllProducts: if both limit and sort query params are sent, only limit is applied (if-else chain, not combined). No 400 returned for invalid input -- missing @Valid on @RequestBody. UserDto returns password field in GET responses -- security flaw. Auth login has no 401 handling for bad credentials. ErrorResponseDto leaks internal path via WebRequest.getDescription(false). No Content-Type validation or API versioning (/v1/products).

```
// ProductController - correct status codes for write operations
@PostMapping
public ResponseEntity<Product> addNewProduct(@RequestBody ProductDto product) {
    Product newProduct = productService.addNewProduct(product);
    return ResponseEntity.status(HttpStatus.CREATED).body(newProduct);
}

@DeleteMapping("/{productId}")
public ResponseEntity<Void> deleteProduct(@PathVariable Long productId) {
    productService.deleteProduct(productId);
    return ResponseEntity.noContent().build();  // 204
}
```

Improved Version: Current -> Better -> Production

Aspect	Current	Better	Production
Validation	None	@Valid + @NotBlank	Custom validators
Errors	Generic 500 catch-all	400 for bad input	RFC 7807 Problem Details
Security	Password in response	@JsonIgnore password	OAuth2 + HTTPS only
Versioning	None	/api/v1 prefix	Header-based versioning

HTTPS	HTTP localhost	Self-signed SSL dev	TLS termination at LB

HTTP Status Codes Used in Project

Code	Meaning	Where
200	OK	GET, PUT, login success
201	Created	POST products/carts/users
204	No Content	DELETE products/carts/users
404	Not Found	ProductNotFoundException, Reso
500	Server Error	GlobalExceptionHandler catch-a
502	Bad Gateway	RestClientResponseException, E

Revision Table

Concept	Rule	productService Example
Idempotent	Same result if repeated	GET, PUT, DELETE
Safe	No side effects	GET only
Resource URL	Noun not verb	/products not /getProducts
JSON body	POST/PUT payload	ProductDto in request
Query params	Filtering/options	?limit=5&sort=asc
Path variable	Resource ID	/products/{productId}

Interview Questions & Answers

Q: Difference between PUT and PATCH?

A: PUT replaces entire resource. PATCH updates partial fields. productService uses PUT only.

Q: When to return 201 vs 200?

A: 201 Created for successful resource creation (POST). 200 OK for reads and updates.

Q: What is idempotency?

A: Multiple identical requests have same effect. DELETE /products/5 twice: second may 404 but resource stays deleted.

Q: What does HTTPS provide?

A: Encryption (confidentiality), integrity, and server authentication via certificates.

Q: What is Content-Type header?

A: Tells server body format. Spring expects application/json for @RequestBody deserialization via Jackson.

Q: Why 204 No Content for DELETE?

A: Resource deleted successfully; no body needed. ProductController.deleteProduct uses noContent().

Q: What is wrong with returning password in UserDto?

A: GET /users exposes passwords to any caller. Must use @JsonIgnore or separate request/response DTOs.

Q: REST vs SOAP?

A: REST uses HTTP+JSON, lightweight. SOAP uses XML envelopes. This project is REST.

Q: What is HATEOAS?

A: Hypermedia links in responses. Not implemented here; responses are plain JSON objects.

Q: How does Spring map HTTP to Java methods?

A: @GetMapping, @PostMapping etc. on controller methods. DispatcherServlet routes by URL pattern and HTTP verb.

HTTPS/TLS Deep Dive

FakeStore API uses HTTPS (TLS 1.2+). RestClient handles certificate validation automatically via Java trust store. Local productService runs plain HTTP on 8080. In production, TLS terminates at load balancer (AWS ALB) or via server.ssl.* properties.

```
HTTP (dev):  Client ----plain text----> localhost:8080
HTTPS (prod): Client --TLS encrypt--> ALB --HTTP--> Spring Boot pod
HTTPS (upstream): RestClient --TLS--> fakestoreapi.com
```

Common HTTP Headers in productService

Header	Used?	Notes
Content-Type: application/json	Auto	Jackson sets on responses
Accept: application/json	Default	Client should send for JSON
Authorization	No	Auth token not validated
User-Agent	Default	Java-http-client sent by RestC

Chapter 5: RestClient & Third-Party APIs

Theory: Beginner to Advanced

Beginner: What Is RestClient?

RestClient is Spring Framework 6.1+ synchronous HTTP client for calling external REST APIs. productService uses it to call fakestoreapi.com. It replaces the older RestTemplate with a fluent API: restClient.get().uri(...).retrieve().body(...).

Intermediate: RestClient vs RestTemplate vs WebClient

RestTemplate: legacy synchronous client, still works but in maintenance mode. RestClient: modern synchronous replacement, fluent builder, same thread blocks until response. WebClient: reactive/non-blocking (Project Reactor), for high concurrency async pipelines. productService correctly uses RestClient for simple proxy calls where blocking I/O is acceptable.

Advanced: retrieve() vs exchange(), Error Handling

retrieve(): simplified API, throws RestClientResponseException on 4xx/5xx. exchange(): lower-level, gives ResponseEntity with full control over status/headers. onStatus(): custom handler before exception. productService uses retrieve() everywhere and catches RestClientResponseException in services to map 404 to domain exceptions.

Why Do We Need an HTTP Client?

productService has no database. All data comes from FakeStore API. RestClient handles connection, serialization (JSON to ProductDto via Jackson), and HTTP protocol. Without it, you would use raw java.net.HttpURLConnection -- verbose and error-prone.

Real-World Analogy

RestClient is a bilingual phone interpreter. Your service speaks Java objects; FakeStore speaks JSON over HTTP. RestClient dials the number (base URL), states the request (GET /products/5), listens to the reply, and translates JSON into ProductDto that ProductMapper converts to Product.

Internal Working

```
FakeStoreProductImplementation
         |
         v
restClient.get()
     .uri("/products/{productId}", productId)
     .retrieve()
     .body(ProductDto.class)
         |
         v
+------------------+
| RestClient       |
| (sync blocking)  |
+--------+---------+
         |
         |  HTTPS GET https://fakestoreapi.com/products/5
         v
+------------------+
```

```
| FakeStore API    |
+--------+---------+
         |
         |  JSON response body
         v
+------------------+
| Jackson          |  Deserialize to ProductDto
+--------+---------+
         |
         v
ProductMapper.toProduct(dto) -> Product

Error path (404):
RestClientResponseException
     -> catch in service
     -> throw ProductNotFoundException
     -> GlobalExceptionHandler -> 404 JSON
```

Practical Example: RestClientConfig and Usage

```
// RestClientConfig.java
@Bean
public RestClient restClient() {
    return RestClient.builder()
            .baseUrl(baseUrl)  // https://fakestoreapi.com
            .build();
}

// FakeStoreProductImplementation.java - GET single product
ProductDto productDto = restClient.get()
        .uri("/products/{productId}", productId)
        .retrieve()
        .body(ProductDto.class);

// POST new product
ProductDto response = restClient.post()
        .uri("/products")
        .body(productDto)
        .retrieve()
        .body(ProductDto.class);

// DELETE - no body in response
restClient.delete()
        .uri("/products/{productId}", productId)
        .retrieve()
        .toBodilessEntity();
```

Issues in Current Code

RestClient has no connect/read timeout -- hung upstream calls block threads forever. No retry on transient failures. ExternalApiException exists but is never thrown; services re-throw RestClientResponseException or rely on GlobalExceptionHandler. Auth login (FakeStoreAuthImplementation) has zero error handling -- bad credentials bubble as raw 4xx. Category and list endpoints do not catch 404/500. Single shared RestClient for all services -- no per-resource configuration.

Improved Version: Current -> Better -> Production

Aspect	Current	Better	Production

Client	Bare RestClient.builder()	Timeouts + default headers	Resilience4j retry
Errors	Ad-hoc try/catch per method	Central RestClient interceptor	Circuit breaker
Auth	No error handling	Map 401 to custom exception	Token refresh logic
Testing	Hits real API	MockRestServiceServer	WireMock in CI
Async	Blocking only	WebClient for slow calls	Virtual threads (Java 21)

retrieve() vs exchange() Example

```
// retrieve() - used in project (simple)
ProductDto dto = restClient.get()
    .uri("/products/1")
    .retrieve()
    .body(ProductDto.class);

// exchange() - more control (not used in project)
ResponseEntity<ProductDto> response = restClient.get()
    .uri("/products/1")
    .exchange((req, res) -> {
        if (res.getStatusCode().is2xxSuccessful()) {
            return res.bodyTo(ProductDto.class);
        }
        throw new ExternalApiException("Failed: " + res.getStatusCode());
    });
```

Revision Table

Method	HTTP Verb	Used In
restClient.get()	GET	All read operations
restClient.post()	POST	Create product/cart/user/login
restClient.put()	PUT	Update product/cart/user
restClient.delete()	DELETE	Delete product/cart/user
.retrieve()	Execute request	All service methods
.body(Class)	Deserialize JSON	ProductDto, CartDto, etc.
.toBodilessEntity()	No body expected	DELETE operations

Interview Questions & Answers

Q: Why RestClient over RestTemplate?

A: RestClient is the modern fluent API, better ergonomics, RestTemplate is in maintenance mode since Spring 5.

Q: When to use WebClient?

A: Non-blocking reactive apps, many concurrent outbound calls, Spring WebFlux stack. Overkill for simple sync proxy like productService.

Q: What is RestClientResponseException?

A: Thrown by retrieve() on error status. Contains getStatusCode(), getResponseBodyAsString(). Caught in product services.

Q: How is base URL configured?

A: RestClientConfig @Bean with .baseUrl() from @Value fakestore.api.base-url. Relative URIs like /products appended to base.

Q: What does .uri("/products/{id}", id) do?

A: URI template expansion. {id} replaced with actual productId value.

Q: How to handle 404 from external API?

A: Catch RestClientResponseException, check status == 404, throw ProductNotFoundException. Done in getSingleProduct.

Q: Why is ExternalApiException unused?

A: Defined and handled in GlobalExceptionHandler but no service throws it. Dead code path; services should use it for clarity.

Q: What is missing for production RestClient?

A: Timeouts, connection pooling config, retry, circuit breaker, request/response logging.

Q: How does Jackson deserialization work?

A: Spring auto-configures ObjectMapper. RestClient uses it to map JSON fields to ProductDto properties (image -> image).

Q: Can one RestClient call multiple APIs?

A: Technically yes but bad practice. Each base URL should have its own RestClient @Bean.

Chapter 6: Exception Handling

Theory: Beginner to Advanced

Beginner: Why Exception Handling Matters

Without centralized handling, every controller method needs try-catch blocks. Clients would get ugly 500 stack traces or inconsistent error formats. productService uses @RestControllerAdvice in GlobalExceptionHandler to convert exceptions into structured ErrorResponseDto JSON with proper HTTP status codes.

Intermediate: @ExceptionHandler and Custom Exceptions

@ExceptionHandler marks methods that handle specific exception types. ProductNotFoundException is thrown when FakeStore returns 404 for a product. ResourceNotFoundException is generic (Cart, User). Both map to 404. RestClientResponseException catches unhandled upstream errors. Exception.class is the catch-all for everything else -> 500.

Advanced: Exception Handler Priority and Design

Spring matches the most specific handler first. Order matters when hierarchies overlap. @RestControllerAdvice applies to all @RestController beans. Production systems add ProblemDetail (RFC 7807), correlation IDs, and avoid exposing internal paths. ErrorResponseDto includes timestamp, message, status, path.

Why Do We Need Global Exception Handling?

FakeStoreProductImplementation catches 404 locally and throws ProductNotFoundException. Without GlobalExceptionHandler, Spring would return default error HTML or empty 500. Clients get predictable JSON: {"message":"Product with ID 99 not found", "status":404,"path":"uri=/products/99","timestamp":"..."}.

Real-World Analogy

GlobalExceptionHandler is the hotel front desk complaint department. Individual staff (controllers) escalate problems (throw exceptions). The desk (advice class) follows a script (ErrorResponseDto format) to respond consistently whether the issue is a missing room key (404) or a power outage (500).

Internal Working

```
Request: GET /products/999
         |
         v
ProductController.getSingleProduct(999)
         |
         v
FakeStoreProductImplementation.getSingleProduct(999)
         |
         |  RestClient -> FakeStore returns 404
         v
catch RestClientResponseException (status 404)
         |
         v
throw new ProductNotFoundException(999)
         |
         |  (bubbles up, no catch in controller)
```

```
         v
GlobalExceptionHandler.handleProductNotFound()
         |
         v
ResponseEntity<ErrorResponseDto> status 404
{
  "message": "Product with ID 999 not found",
  "status": 404,
  "path": "uri=/products/999",
  "timestamp": "2026-08-02T..."
}
```

Practical Example: productService Exception Classes

```
// ProductNotFoundException.java
public class ProductNotFoundException extends RuntimeException {
    public ProductNotFoundException(Long productId) {
        super("Product with ID " + productId + " not found");
    }
}

// GlobalExceptionHandler.java
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ProductNotFoundException.class)
    public ResponseEntity<ErrorResponseDto> handleProductNotFound(
            ProductNotFoundException ex, WebRequest request) {
        ErrorResponseDto error = new ErrorResponseDto(
            ex.getMessage(), HttpStatus.NOT_FOUND.value(),
            request.getDescription(false));
        return new ResponseEntity<>(error, HttpStatus.NOT_FOUND);
    }

    @ExceptionHandler(Exception.class)
    public ResponseEntity<ErrorResponseDto> handleGeneric(
            Exception ex, WebRequest request) {
        ErrorResponseDto error = new ErrorResponseDto(
            "Internal server error", 500, request.getDescription(false));
        return new ResponseEntity<>(error, HttpStatus.INTERNAL_SERVER_ERROR);
    }
}
```

Issues in Current Code

handleGeneric catches ALL Exception including potentially recoverable ones -- masks real bugs and logs nothing (no @Slf4j logging). Auth login failures not mapped to 401. List endpoints (getAllProducts, getAllCarts) have no 404 handling because they do not throw on empty results. ExternalApiException handler exists but nothing throws it. ProductNotFoundException and ResourceNotFoundException duplicate logic -- could use one NotFoundException with resource type. Error message 'Internal server error' hides actual exception for debugging.

Improved Version: Current -> Better -> Production

Aspect	Current	Better	Production
Logging	None	Slf4j log in each handler	Structured JSON logs
Validation	No handler	MethodArgumentNotValidExceptio	Field-level errors
Auth errors	Unhandled	InvalidCredentialsException ->	OAuth2 error format

Catch-all	Hides stack trace	Log ex, return generic msg	Sentry alert on 500
Response	Custom DTO	RFC 7807 ProblemDetail	Correlation ID header

Exception Mapping Table

Exception	HTTP Status	Thrown By
ProductNotFoundException	404	FakeStoreProductImplementation
ResourceNotFoundException	404	Cart/User services
RestClientResponseException	varies/502	Unhandled upstream errors
ExternalApiException	502	Never thrown (dead code)
Exception	500	Any unhandled error

Revision Table

Annotation	Purpose	In Project
@RestControllerAdvice	Global exception handling	GlobalExceptionHandler
@ExceptionHandler	Map exception to response	Per exception type
RuntimeException	Unchecked, no throws clause	All custom exceptions
ErrorResponseDto	Standard error body	message, status, path, timesta
WebRequest	Request metadata	getDescription for path

Interview Questions & Answers

Q: What is @ControllerAdvice vs @RestControllerAdvice?
A: @RestControllerAdvice = @ControllerAdvice + @ResponseBody on all handlers. Use RestControllerAdvice for REST APIs returning JSON.

Q: Why throw custom exceptions instead of returning null?
A: Exceptions separate error flow from happy path. Controller stays clean; handler centralizes error response format.

Q: Where should RestClient 404 be handled?
A: Service layer catches and throws domain exception. Handler converts to HTTP 404. Keeps HTTP concerns out of service interface.

Q: What is wrong with @ExceptionHandler(Exception.class)?
A: Too broad; swallows all errors as generic 500 without logging. Makes debugging production issues difficult.

Q: Checked vs unchecked exceptions in Spring?
A: Spring MVC only handles unchecked exceptions from controllers by default. All project exceptions extend RuntimeException.

Q: How to return 400 for validation errors?
A: Add @Valid on @RequestBody, handle MethodArgumentNotValidException in GlobalExceptionHandler. Not implemented yet.

Q: Difference between ProductNotFoundException and ResourceNotFoundException?
A: Product-specific vs generic. Functionally identical (both 404). Redundant design; one class with resource name would suffice.

Q: What does request.getDescription(false) return?
A: Request URI description like uri=/products/5. The false param excludes session id.

Q: Should services return Optional instead of throwing?
A: For not-found cases, exceptions are idiomatic in Spring MVC. Optional leads to ambiguous empty 200 responses.

Q: How to test exception handlers?

A: @WebMvcTest with MockMvc, trigger endpoint that throws exception, assert JSON body and status code.

Error Response Structure

```
// ErrorResponseDto returned on errors
{
  "message": "Product with ID 99 not found",
  "status": 404,
  "path": "uri=/products/99",
  "timestamp": "2026-08-02T21:30:00"
}

// Problems with current structure:
// 1. path exposes internal URI format
// 2. No error code field for client handling
// 3. No field-level errors for validation
// 4. timestamp uses server local time, not UTC
```

Exception Handling Checklist

For each new endpoint, verify: (1) 404 mapped for missing resources, (2) 400 for bad input once @Valid added, (3) 502 for upstream failures, (4) exceptions logged with stack trace server-side, (5) no sensitive data in response.

Chapter 7: Spring Request Flow

Theory: Beginner to Advanced

Beginner: What Happens When You Call an API?

When a client sends GET http://localhost:8080/products/5, the embedded Tomcat server receives the HTTP request and passes it to Spring's DispatcherServlet -- the front controller for all Spring MVC requests.

Intermediate: HandlerMapping and HandlerAdapter

DispatcherServlet asks HandlerMapping to find which controller method handles /products/5 + GET. RequestMappingHandlerMapping matches ProductController.getSingleProduct with @GetMapping("/{productId}"). HandlerAdapter invokes the method, resolves @PathVariable productId=5, calls productService.getSingleProduct(5), wraps result in ResponseEntity, and HttpMessageConverter (Jackson) serializes Product to JSON.

Advanced: Full Filter Chain and View Resolution

Before DispatcherServlet: Servlet Filter chain (encoding, security if configured). After handler: ReturnValueHandler processes ResponseEntity. For REST, no view resolver needed -- MappingJackson2HttpMessageConverter writes JSON directly. If ProductNotFoundException thrown, DispatcherServlet delegates to ExceptionHandlerExceptionResolver -> GlobalExceptionHandler.

Why Understand Request Flow?

Debugging 404 vs 405 vs 500 requires knowing where routing fails. If URL is wrong, HandlerMapping returns 404 before controller runs. If method is POST on GET-only endpoint, 405 Method Not Allowed. If service throws unhandled exception, ExceptionHandlerExceptionResolver or default handling kicks in.

Real-World Analogy

DispatcherServlet is an airport traffic control tower. HandlerMapping is the flight schedule (which gate for which flight). The controller is the gate agent. HandlerAdapter is the protocol for talking to the agent. MessageConverter is baggage claim formatting your luggage (Java object) into JSON suitcase.

Internal Working: Complete Flow

```
HTTP GET /products/5
     |
     v
[Tomcat Connector]
     |
     v
[Filter Chain]  (encoding, etc.)
     |
     v
+--------------------------+
|   DispatcherServlet      |
+--------------------------+
     |
     | 1. HandlerMapping lookup
     v
RequestMappingHandlerMapping
```

```
-> ProductController.getSingleProduct(@PathVariable 5)
     |
     | 2. HandlerAdapter invokes
     v
ProductController
     |
     v
FakeStoreProductImplementation.getSingleProduct(5)
     |
     v
RestClient -> FakeStore API -> ProductDto -> ProductMapper -> Product
     |
     | 3. Return value handling
     v
ResponseEntity<Product> (200 OK)
     |
     | 4. HttpMessageConverter
     v
Jackson -> JSON response body
     |
     v
HTTP/1.1 200 OK
Content-Type: application/json
{"id":5,"title":"...","price":...}
```

Practical Example: Annotating the Flow

```
// Step 1: Class-level mapping
@RestController
@RequestMapping("/products")          // base path
public class ProductController {

    // Step 2: Method-level mapping
    @GetMapping("/{productId}")       // GET /products/{productId}
    public ResponseEntity<Product> getSingleProduct(
            @PathVariable Long productId) {   // Step 3: arg resolution
        Product product = productService.getSingleProduct(productId);
        return ResponseEntity.ok(product);    // Step 4: wrap response
    }
}

// Spring resolves:
// URL: /products/5 -> productId = 5
// Accept: application/json -> Jackson converter selected
// Return type: ResponseEntity<Product> -> status 200 + body
```

Issues in Current Code

No Spring Security filter in chain -- all endpoints publicly accessible. No request logging filter or MDC correlation ID. Actuator endpoints may be exposed without security (dependency present, not configured). ProductController returns Product model directly exposing internal structure instead of response DTO. No interceptors for timing or auth.

Improved Version: Current -> Better -> Production

Layer	Current	Better	Production
Filters	Default only	Request logging filter	Auth + rate limit filters
Interceptors	None	HandlerInterceptor for timing	Distributed tracing
Resolution	Default converters	Custom date formatting	Content negotiation

Errors	ControllerAdvice only	Filter-level error handling	API gateway errors
Docs	None	springdoc at /swagger-ui	AsyncAPI for events

Key Components Table

Component	Role	productService
DispatcherServlet	Front controller	Auto-configured by webmvc star
HandlerMapping	URL to handler	Maps /products/* to ProductCon
HandlerAdapter	Invoke handler method	RequestMappingHandlerAdapter
HttpMessageConverter	Java <-> HTTP body	MappingJackson2HttpMessageConv
ExceptionResolver	Handle exceptions	ExceptionHandlerExceptionResol
RestClient	Outbound HTTP	Separate from inbound MVC flow

Revision Table

Annotation	Resolved By	Example
@PathVariable	Path segment	productId from /products/5
@RequestParam	Query string	limit, sort in GET /products
@RequestBody	Request body JSON	ProductDto on POST
@RequestMapping	Class/method URL	/products base path
ResponseEntity	Status + headers + body	201 Created, 204 No Content

Interview Questions & Answers

Q: What is DispatcherServlet?

A: Spring MVC front controller servlet that dispatches requests to appropriate handlers and renders responses.

Q: How does Spring map /products/5 to a method?

A: Combines class @RequestMapping with method @GetMapping, uses @PathVariable to bind 5 to productId.

Q: What converts Product object to JSON?

A: MappingJackson2HttpMessageConverter using Jackson ObjectMapper. Auto-configured with webmvc starter.

Q: What happens if no handler matches?

A: DispatcherServlet throws NoHandlerFoundException -> 404 (if configured) or default servlet 404.

Q: Where does exception handling fit in the flow?

A: After handler throws, ExceptionHandlerExceptionResolver finds @ExceptionHandler in @RestControllerAdvice.

Q: Difference between Filter and Interceptor?

A: Filters are servlet-level (before DispatcherServlet). Interceptors are Spring MVC level (around handler execution).

Q: What is HandlerMapping order?

A: Multiple mappings exist; @RequestMapping on controller is evaluated with most specific match winning.

Q: How are query params bound?

A: @RequestParam on method parameter. GET /products?limit=5 binds limit=5. Both optional in getAllProducts.

Q: What is the role of Tomcat here?

A: Embedded servlet container hosting DispatcherServlet. Started by Spring Boot auto-configuration.

Q: Inbound vs outbound flow in productService?

A: Inbound: HTTP -> Controller -> Service. Outbound: Service -> RestClient -> FakeStore. Two separate HTTP stacks.

Request Flow for POST /products

```
POST /products + JSON body
     |
     v
DispatcherServlet -> ProductController.addNewProduct(@RequestBody ProductDto)
     |
     v
FakeStoreProductImplementation.addNewProduct(dto)
     |
     v
restClient.post().uri("/products").body(dto).retrieve().body(ProductDto.class)
     |
     v
ProductMapper.toProduct(response) -> Product
     |
     v
ResponseEntity.status(201).body(product)
     |
     v
Jackson serializes Product -> JSON response (201 Created)
```

Exception Flow Overlay

If RestClient throws during any step, exception propagates past controller. ExceptionHandlerExceptionResolver intercepts before default error handling. GlobalExceptionHandler produces ErrorResponseDto. Client never sees Java stack trace.

Chapter 8: Product Service Architecture

Theory: Beginner to Advanced

Beginner: Project Structure

productService follows a layered package structure under org.pavan.productservice: controllers (HTTP), services (business + external calls), dtos (API data shapes), models (domain objects), mappers (conversion), config (beans), exceptions (errors). No repository or database layer -- FakeStore is the data source.

Intermediate: Layer Responsibilities

Controllers: thin, delegate to services, return ResponseEntity. Services: implement interfaces (ProductService, CartService, etc.), call RestClient, map errors. DTOs: match external JSON (ProductDto.image maps to FakeStore 'image' field). Models: internal representation (Product.imageUrl). Mappers: bridge DTO-model gap. Config: RestClient bean with externalized base URL.

Advanced: Design Patterns in Use

Proxy/Gateway: entire app proxies FakeStore. Strategy: service interfaces allow swapping implementations. Adapter: ProductMapper adapts FakeStore JSON shape to domain model. Facade: controllers expose simplified API over multiple FakeStore endpoints. Missing patterns: Repository, Factory, Observer, Circuit Breaker.

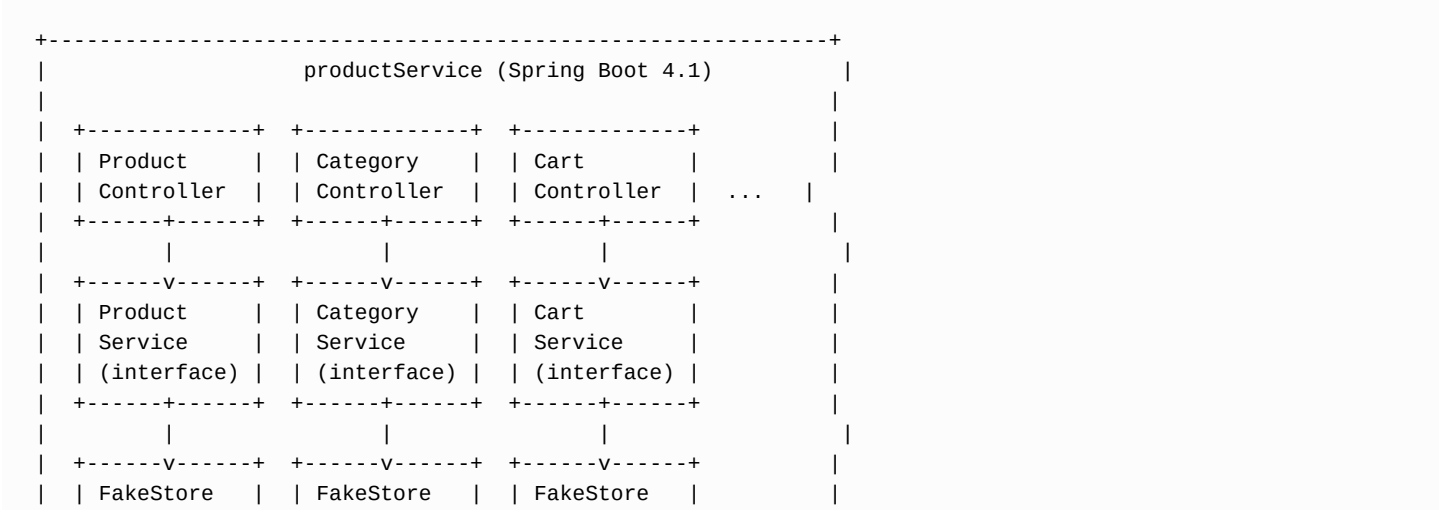
Why This Architecture?

Separation lets you change FakeStore integration without touching controllers. Adding a database later means new service implementations, not controller rewrites. GlobalExceptionHandler provides cross-cutting error handling without polluting every layer.

Real-World Analogy

The architecture is a translation bureau. Controllers are receptionists taking requests in English (your API). Services are translators calling the foreign office (FakeStore). DTOs are the foreign language documents. Models are the internal filing system. Mappers are bilingual staff converting between formats.

Architecture Diagram



```
|  | ProductImpl |  | CategoryImpl|  | CartImpl    |          |
|  +------+------+  +------+------+  +------+------+          |
|         |                |                |                  |
|         +----------------+----------------+                  |
|                          |                                   |
|                   +------v------+                            |
|                   | RestClient  |  RestClientConfig          |
|                   +------+------+                            |
|                          |                                   |
|  +-----------------------v-------------------------------+   |
|  | GlobalExceptionHandler | ProductMapper | DTOs/Models  |   |
|  +-------------------------------------------------------+   |
+-----------------------------|-------------------------------+
                              |
                              v
                  https://fakestoreapi.com
```

Package Inventory

Package	Files	Responsibility
controllers	5 classes	REST endpoints
services	5 interfaces + 5 impls	Business + RestClient
dtos	12 classes	JSON serialization shapes
models	3 classes	Domain objects
mappers	1 class	Product DTO <-> Model
config	1 class	RestClient bean
exceptions	5 classes	Errors + global handler

Issues in Current Architecture

Inconsistent return types: ProductController returns Product model, CategoryController returns ProductDto list. No mapper for Cart/User -- DTOs returned directly from FakeStore. UserDto includes password with no separation of request/response DTOs. Main2.java pollutes root package. BaseModel has unused fields (createdAt, isDeleted) never populated. RatingDto in ProductDto ignored by ProductMapper. AuthService calls /auth/login on FakeStore which may not be a real FakeStore endpoint. Single RestClient shared across all domains with no abstraction.

Improved Version: Current -> Better -> Production

Layer	Current	Better	Production
Controller	Mixed Model/DTO returns	Response DTOs only	OpenAPI contracts
Service	Concrete FakeStore impl	Interface + mock + real	Hexagonal ports
Mapper	Product only	Mappers for all entities	MapStruct codegen
Config	One RestClient	Typed API clients	Service mesh
Cross-cut	Exception handler only	+ Validation + Logging	+ Metrics + Tracing

Dependency Graph

ProductController -> ProductService <- FakeStoreProductImplementation -> RestClient <- RestClientConfig. Same pattern for Cart, User, Category, Auth. GlobalExceptionHandler is independent, scanned by component scan. ProductMapper is static utility called from FakeStoreProductImplementation only.

```
// Layer violation example (inconsistency, not strict violation):
```

```
// ProductController returns domain Model:
public ResponseEntity<Product> getSingleProduct(...) { ... }

// CategoryController returns DTO:
public ResponseEntity<List<ProductDto>> getProductsInCategory(...) { ... }

// Correct approach: all controllers return response DTOs,
// services return domain models or DTOs consistently.
```

Revision Table

Pattern	Status	Notes
Layered Architecture	Implemented	Controller-Service-External
Dependency Injection	Implemented	Constructor injection througho
DTO Pattern	Partial	Products mapped; carts/users r
Repository Pattern	Missing	No database layer
BFF/Gateway	Implicit	Proxies FakeStore
Interface Segregation	Partial	Service interfaces exist

Interview Questions & Answers

Q: Describe productService architecture.
A: Layered Spring Boot app: 5 controllers, 5 service interfaces with FakeStore implementations, shared RestClient, global exception handling, ProductMapper for product DTO conversion.

Q: Why separate DTOs and models?
A: DTOs match external API contract (image field). Models represent internal domain (imageUrl). Decouples API changes from domain.

Q: What is missing compared to production?
A: Database, auth, validation, tests, logging, caching, API docs, consistent DTO usage, password masking.

Q: Why service interfaces if only one implementation?
A: Enables testing with mocks and future swap to database-backed implementation without controller changes.

Q: What is the role of RestClientConfig?
A: Creates singleton RestClient bean with FakeStore base URL from application.properties.

Q: How many controllers and what do they map to?
A: 5: Product, Category, Cart, User, Auth. Each maps to corresponding FakeStore API resource.

Q: What cross-cutting concerns exist?
A: Only exception handling. Missing: logging, security, validation, transaction management.

Q: Why is ProductMapper static?
A: Utility class with private constructor. Works but not injectable/testable as a Spring bean.

Q: What would hexagonal architecture change?
A: Ports (interfaces) for inbound (controllers) and outbound (FakeStore client) adapters. Clearer test boundaries.

Q: What dead code exists?
A: Main2.java (unrelated main method), ExternalApiException (never thrown), commented Lombok annotations in BaseModel.

Data Flow: Product Read Operation

Tracing data through layers for GET /products/5:

Step	Layer	Type	Transformation
1	FakeStore API	JSON	Raw HTTP response
2	RestClient	ProductDto	Jackson deserialization
3	Service	ProductDto	No transformation
4	ProductMapper	Product	DTO to domain model
5	Controller	Product	Wrap in ResponseEntity
6	Jackson	JSON	Serialize Product to client

Comparison: Product vs Cart Data Flow

Products: FakeStore JSON -> ProductDto -> ProductMapper -> Product -> client. Carts: FakeStore JSON -> CartDto -> client directly. No mapper, no domain model. This inconsistency means cart logic cannot add computed fields without changing DTO.

Chapter 9: API Analysis - All Endpoints

Overview

productService exposes 24 endpoints across 5 controllers, all proxying fakestoreapi.com via RestClient. Base URL: http://localhost:8080. This chapter analyzes every endpoint: method, path, request, response, upstream call, status codes, and identified issues.

ProductController (/products)

GET /products

Returns all products. Optional query params: limit (int), sort (string). Upstream: GET /products, /products?limit=N, or /products?sort=N. Returns List<Product> (domain model, not DTO). No error handling for upstream failures.

Bug

If both limit and sort are provided, only limit is used. Should support combined query or return 400 for conflicting params.

GET /products/{productId}

Returns single product by ID. Upstream: GET /products/{id}. 404 from FakeStore -> ProductNotFoundException -> 404 JSON. Only product endpoint with proper 404 handling.

GET /products/category/{categoryName}

Products filtered by category name (e.g., 'electronics'). Upstream: GET /products/category/{categoryName}. Fixed from earlier ID-based approach -- FakeStore uses category name strings.

POST /products

Creates product. Body: ProductDto. Returns 201 Created with Product. Upstream: POST /products. No validation on input. No error handling for upstream 4xx/5xx beyond global handler.

PUT /products/{productId}

Updates product. Body: ProductDto. Returns 200 with updated Product. Upstream: PUT /products/{id}. 404 -> ProductNotFoundException.

DELETE /products/{productId}

Deletes product. Returns 204 No Content. Upstream: DELETE /products/{id}. 404 -> ProductNotFoundException.

CategoryController (/categories)

GET /categories

Returns all categories as List<CategoryDto> with name field populated. Upstream: GET /products/categories returns String[]. Service manually wraps each string in CategoryDto -- no ID, no description.

GET /categories/{categoryName}/products

Returns products in category as List<ProductDto> (not Product model). Upstream: GET /products/category/{categoryName}. Inconsistent with ProductController which returns Product for same data.

CartController (/carts)

GET /carts

All carts. Upstream: GET /carts. Returns List<CartDto>. No pagination.

GET /carts/{cartId}

Single cart. Upstream: GET /carts/{id}. 404 -> ResourceNotFoundException.

GET /carts/user/{userId}

Carts for user. Upstream: GET /carts/user/{userId}. No 404 if user has no carts.

POST /carts

Create cart. Body: CartDto. Returns 201. Upstream: POST /carts.

PUT /carts/{cartId}

Update cart. 404 handled. Upstream: PUT /carts/{id}.

DELETE /carts/{cartId}

Delete cart. Returns 204. 404 handled. Upstream: DELETE /carts/{id}.

UserController (/users)

GET /users

All users. Upstream: GET /users. Returns passwords in response -- security issue.

GET /users/{userId}

Single user. 404 handled. Password exposed in response.

POST /users

Create user. Returns 201. Password sent to FakeStore and returned.

PUT /users/{userId}

Update user. 404 handled.

DELETE /users/{userId}

Delete user. Returns 204. 404 handled.

AuthController (/auth)

POST /auth/login

Login with LoginRequestDto (username, password). Returns LoginResponseDto (token). Upstream: POST /auth/login. FakeStore may not support this endpoint reliably. No error handling -- invalid credentials produce raw upstream error.

Complete Endpoint Matrix

#	Method	Path	Status	Issues
1	GET	/products	200	limit/sort conflict
2	GET	/products/{id}	200/404	OK
3	GET	/products/category/{name}	200	No 404 for bad category
4	POST	/products	201	No validation
5	PUT	/products/{id}	200/404	No validation
6	DELETE	/products/{id}	204/404	OK
7	GET	/categories	200	Manual DTO wrapping
8	GET	/categories/{name}/products	200	Returns DTO not Model
9	GET	/carts	200	No pagination
10	GET	/carts/{id}	200/404	OK
11	GET	/carts/user/{id}	200	OK
12	POST	/carts	201	No validation
13	PUT	/carts/{id}	200/404	OK
14	DELETE	/carts/{id}	204/404	OK
15	GET	/users	200	Password leak
16	GET	/users/{id}	200/404	Password leak
17	POST	/users	201	Password leak
18	PUT	/users/{id}	200/404	Password leak
19	DELETE	/users/{id}	204/404	OK
20	POST	/auth/login	200	No error handling

Improved Version: Current -> Better -> Production

Issue	Current	Better	Production
Return types	Mixed Model/DTO	Uniform response DTOs	Versioned schemas
Validation	None	@Valid on all POST/PUT	JSON Schema validation
Security	Passwords exposed	@JsonIgnore + separate DTOs	OAuth2 + RBAC
Pagination	None on lists	Pageable param	Cursor-based pagination
Errors	Inconsistent 404	404 for all not-found	Standard error RFC 7807

Revision Table

Controller	Endpoints	Upstream Base
ProductController	6	/products
CategoryController	2	/products/categories
CartController	6	/carts
UserController	5	/users
AuthController	1	/auth/login

Interview Questions & Answers

Q: How many total endpoints does productService expose?

A: 20 endpoints across 5 controllers (6 product + 2 category + 6 cart + 5 user + 1 auth).

Q: Which endpoints have proper 404 handling?

A: GET/PUT/DELETE by ID for products, carts, and users. List endpoints and category lookups do not.

Q: What is inconsistent about return types?

A: ProductController returns Product model; CategoryController.getProductsInCategory returns ProductDto.

Q: What FakeStore endpoint does GET /categories call?

A: GET /products/categories which returns array of category name strings.

Q: What is the limit/sort bug in GET /products?

A: if-else checks limit first; if both query params sent, sort is silently ignored.

Q: Why is GET /users a security problem?

A: UserDto includes password field; FakeStore returns it and project passes it through to clients.

Q: Does FakeStore support /auth/login?

A: FakeStore API documentation focuses on products/carts/users. Auth endpoint may not work as expected.

Q: What HTTP status does DELETE return?

A: 204 No Content for products, carts, users.

Q: What was fixed about category endpoints?

A: Changed from category ID to category name in path, matching FakeStore API contract.

Q: Which endpoints lack input validation?

A: All POST and PUT endpoints -- no @Valid, no @NotNull/@NotBlank on DTO fields.

Chapter 10: Missing APIs - Implementation

Context

The original productService project had stub controllers and incomplete service methods. The current codebase implements full CRUD for products, carts, users, and auth login. This chapter documents what was missing, what was implemented, and what remains problematic.

Previously Missing: Product PUT and DELETE

Early versions only had GET and POST for products. Current FakeStoreProductImplementation now implements updateProduct and deleteProduct using RestClient PUT and DELETE.

```
// FakeStoreProductImplementation.java - now implemented
@Override
public Product updateProduct(Long productId, ProductDto productDto) {
    try {
        ProductDto response = restClient.put()
                .uri("/products/{productId}", productId)
                .body(productDto)
                .retrieve()
                .body(ProductDto.class);
        return ProductMapper.toProduct(response);
    } catch (RestClientResponseException ex) {
        if (ex.getStatusCode().value() == 404) {
            throw new ProductNotFoundException(productId);
        }
        throw ex;
    }
}

@Override
public void deleteProduct(Long productId) {
    try {
        restClient.delete()
                .uri("/products/{productId}", productId)
                .retrieve()
                .toBodilessEntity();
    } catch (RestClientResponseException ex) {
        if (ex.getStatusCode().value() == 404) {
            throw new ProductNotFoundException(productId);
        }
        throw ex;
    }
}
```

Remaining Issue

PUT sends entire ProductDto but FakeStore may not persist changes (it is a fake API). No partial update (PATCH) support.

Previously Missing: Cart CRUD

CartController now has full CRUD: GET all, GET by ID, GET by user, POST, PUT, DELETE. FakeStoreCartImplementation proxies all operations to FakeStore /carts endpoints.

Endpoint	Method	Implementation Status	Issues

GET /carts	getAllCarts	Implemented	No pagination
GET /carts/{id}	getCartById	Implemented	404 handled
GET /carts/user/{id}	getCartsByUserId	Implemented	OK
POST /carts	createCart	Implemented	No validation
PUT /carts/{id}	updateCart	Implemented	404 handled
DELETE /carts/{id}	deleteCart	Implemented	404 handled

Previously Missing: User CRUD

UserController implements full CRUD via FakeStoreUserImplementation. Critical flaw: password field flows through all operations.

```
// UserDto.java - password should NOT be in response
@Getter @Setter
public class UserDto {
    private long id;
    private String email;
    private String username;
    private String password;  // LEAKED in GET responses
    private NameDto name;
    private AddressDto address;
    private long phone;
}

// Fix: create UserResponseDto without password field
// Or: @JsonProperty(access = JsonProperty.Access.WRITE_ONLY) on password
```

Previously Missing: Auth Login

AuthController and FakeStoreAuthImplementation added POST /auth/login. Proxies to FakeStore /auth/login. LoginRequestDto has username/password; LoginResponseDto returns token string.

Issues with Auth Implementation

No try-catch for failed login -- RestClientResponseException goes to global handler as generic external API error, not 401 Unauthorized. No token validation on subsequent requests. No Spring Security integration. FakeStore auth endpoint may return unexpected responses. Token is forwarded as-is with no JWT parsing.

Category Endpoint Fix

Category endpoints were corrected to use category name (String) instead of numeric ID. FakeStore API uses /products/category/{categoryName} where name is a lowercase string like 'jewelery' or 'electronics'.

```
// ProductController - uses category name
@GetMapping("/category/{categoryName}")
public ResponseEntity<List<Product>> getProductsByCategory(
        @PathVariable String categoryName) {
    return ResponseEntity.ok(productService.getProductsByCategory(categoryName));
}

// CategoryController - also uses name
@GetMapping("/{categoryName}/products")
public ResponseEntity<List<ProductDto>> getProductsInCategory(
        @PathVariable String categoryName) {
    return ResponseEntity.ok(categoryService.getProductsInCategory(categoryName));
}
```

What Is Still Missing

1. Input validation (@Valid) on all write endpoints. 2. PATCH partial updates for products/users/carts. 3. Pagination on list endpoints. 4. Search/filter beyond category. 5. Authentication middleware protecting endpoints. 6. Rate limiting. 7. Integration tests for all new endpoints. 8. Separate request/response DTOs for users. 9. Product rating mapping (RatingDto ignored by ProductMapper). 10. Actuator health check configuration.

Improved Version: Current -> Better -> Production

Feature	Was Missing	Now	Still Needed
Product PUT/DELETE	Yes	Implemented	PATCH, validation
Cart CRUD	Yes	Implemented	Validation, pagination
User CRUD	Yes	Implemented	Password masking
Auth login	Yes	Stub-quality	Spring Security JWT
Category by name	Wrong (ID)	Fixed	Category validation

Internal Flow: New Cart Creation

```
POST /carts  {"userId":1, "products":[...]}
     |
     v
CartController.createCart(CartDto)
     |
     v
FakeStoreCartImplementation.createCart(dto)
     |
     v
restClient.post().uri("/carts").body(dto).retrieve()
     |
     v
FakeStore creates cart, returns CartDto with id
     |
     v
ResponseEntity.status(201).body(created)
```

Revision Table

API Area	Endpoints Added	Service Class
Products	PUT, DELETE	FakeStoreProductImplementation
Carts	All 6 endpoints	FakeStoreCartImplementation
Users	All 5 endpoints	FakeStoreUserImplementation
Auth	POST /auth/login	FakeStoreAuthImplementation
Categories	Name-based fix	FakeStoreCategoryImplementatio

Interview Questions & Answers

Q: What APIs were originally stubs?

A: Cart, User, Auth controllers existed with empty or incomplete service methods. Product lacked PUT/DELETE.

Q: How was category endpoint fixed?

A: Changed path variable from numeric ID to String categoryName matching FakeStore API convention.

Q: What is still wrong with user endpoints?

A: Password returned in GET responses. No separate CreateUserRequest vs UserResponse DTOs.

Q: How does cart creation work?

A: CartController POST -> FakeStoreCartImplementation -> RestClient POST /carts -> returns created CartDto with 201 status.

Q: Does auth actually secure the API?

A: No. Login returns a token but no endpoint checks it. No Spring Security filter chain configured.

Q: What validation is missing on new endpoints?

A: All POST/PUT bodies accept any JSON without @Valid constraints.

Q: How is product delete different from get for error handling?

A: Both catch 404 and throw ProductNotFoundException. Delete uses toBodilessEntity().

Q: Can you update a cart's items?

A: PUT /carts/{id} sends full CartDto including products list to FakeStore.

Q: What FakeStore endpoints do new features call?

A: /carts, /users, /auth/login in addition to existing /products endpoints.

Q: What would you implement next?

A: Password masking, @Valid, Spring Security, MockRestServiceServer tests, and pagination on list endpoints.

Chapter 11: Advanced Topics & Best Practices

Theory: Beginner to Advanced

DTO vs Entity (Model)

DTO (Data Transfer Object): shapes data for API transport. ProductDto matches FakeStore JSON with fields like 'image' and flat 'category' string. Entity/Model: internal domain representation. Product has Category object and imageUrl field. ProductMapper converts between them. CartDto and UserDto are used directly without model conversion -- inconsistent approach.

SOLID Principles Applied to productService

S - Single Responsibility: controllers handle HTTP only (mostly followed). O - Open/Closed: service interfaces allow extension (FakeStore impl). L - Liskov: implementations honor interface contracts. I - Interface Segregation: separate service per domain (good). D - Dependency Inversion: controllers depend on ProductService interface (good). Violations: ProductMapper static utility, GlobalExceptionHandler catches all Exception.

Layered Architecture, Validation, Caching, Pagination

Layers: Controller -> Service -> External API (no repository). Validation: not implemented; should use @Valid, @NotNull, @Size on DTOs. Caching: not implemented; @Cacheable could cache GET /products and /categories. Pagination: not implemented; FakeStore supports limit param but no page/offset.

Why These Topics Matter

Without DTO separation, API changes break clients. Without validation, malformed requests reach FakeStore causing cryptic errors. Without caching, every product listing hits external API adding latency. Without pagination, large datasets overload memory and response size.

Real-World Analogy

DTO vs Entity is like a restaurant menu vs kitchen prep. Menu (DTO) shows customer-friendly names and prices. Kitchen (Entity/Model) tracks inventory, suppliers, and prep instructions. Customers never see the walk-in freezer.

DTO vs Entity Flow

```
Inbound (POST /products):
Client JSON -> ProductDto (@RequestBody)
     -> FakeStoreProductImplementation
     -> RestClient sends ProductDto to FakeStore
     -> Response ProductDto
     -> ProductMapper.toProduct() -> Product
     -> Returned to client as Product JSON

Problem: Client sees internal Model, not DTO.
Better: Always return ProductResponseDto to client.

User/Cart flow (no mapper):
Client JSON -> CartDto -> FakeStore -> CartDto -> Client
(DTO used at all layers -- simpler but no domain model)
```

Practical Example: ProductMapper

```
// ProductMapper.java - manual mapping
public static Product toProduct(ProductDto dto) {
    Product product = new Product();
    product.setId(dto.getId());
    product.setTitle(dto.getTitle());
    product.setPrice(dto.getPrice() != null ? dto.getPrice() : 0.0);
    product.setImageUrl(dto.getImage());  // field name mismatch handled here

    if (dto.getCategory() != null) {
        Category category = new Category();
        category.setName(dto.getCategory());  // String -> Object
        product.setCategory(category);
    }
    return product;
}
// RatingDto in ProductDto is NEVER mapped to Product -- data loss
```

Issues in Current Code

No @Valid anywhere. UserDto password exposed. No caching despite read-heavy workload. No pagination on GET /products, /carts, /users. ProductMapper ignores rating. BaseModel fields (createdAt, isDeleted) never used. No MapStruct. SOLID violation: GlobalExceptionHandler has too many responsibilities. No logging framework usage despite SLF4J being on classpath via Spring Boot.

Improved Version: Current -> Better -> Production

Current

Manual ProductMapper, no validation, no caching, no pagination, mixed DTO/Model returns, passwords in responses.

Better

```
// Validation example
public class ProductDto {
    @NotBlank private String title;
    @Positive private Double price;
    @NotBlank private String description;
}

@PostMapping
public ResponseEntity<ProductResponseDto> addNewProduct(
        @Valid @RequestBody ProductDto product) { ... }

// Caching example
@Cacheable("products")
public List<Product> getAllProducts() { ... }

// Pagination example
@GetMapping
public ResponseEntity<Page<ProductResponseDto>> getProducts(
        @RequestParam(defaultValue = "0") int page,
        @RequestParam(defaultValue = "20") int size) { ... }
```

Production

MapStruct for all mappers. Redis cache with TTL. Cursor pagination. Bean Validation + custom validators. Separate command/query DTOs. Structured logging with correlation IDs. API rate limiting.

Topic	Current	Better	Production

DTO/Entity	Partial mapping	All entities mapped	CQRS separation
SOLID	Mostly OK	Split exception handlers	Hexagonal architecture
Validation	None	@Valid on writes	Custom business validators
Caching	None	@Cacheable reads	Redis + cache invalidation
Pagination	limit param only	Pageable	Cursor + total count

SOLID Checklist for productService

Principle	Score	Evidence
Single Responsibility	Medium	Controllers thin; handler over
Open/Closed	Good	Service interfaces extensible
Liskov Substitution	Good	Impls honor contracts
Interface Segregation	Good	Separate service per domain
Dependency Inversion	Good	Constructor injection of inter

Revision Table

Concept	Definition	Project Status
DTO	API data shape	ProductDto, CartDto, UserDto
Entity/Model	Domain object	Product, Category
Mapper	DTO <-> Entity	ProductMapper only
@Valid	Input validation	Not used
@Cacheable	Method-level cache	Not used
Pageable	Spring pagination	Not used

Interview Questions & Answers

Q: DTO vs Entity difference?

A: DTO is for API transport with serialization concerns. Entity represents business domain. ProductDto.image vs Product.imageUrl.

Q: Why not expose Entity directly?

A: Hides internal structure, prevents lazy-loading issues (N/A here), allows different API vs domain shapes.

Q: Which SOLID principle does service interface demonstrate?

A: Dependency Inversion: high-level ProductController depends on ProductService abstraction.

Q: How to add validation?

A: @Valid on @RequestBody + constraint annotations on DTO fields + MethodArgumentNotValidException handler.

Q: How to add caching for products?

A: @EnableCaching + @Cacheable on getAllProducts() + Redis/Ehcache provider.

Q: What pagination options exist?

A: FakeStore supports ?limit=N. Spring Pageable for database. For proxy, pass page/size as query params to upstream.

Q: What data does ProductMapper lose?

A: RatingDto (rate, count) from ProductDto is never mapped to Product model.

Q: Why is password in UserDto a DTO design flaw?

A: Same DTO used for input and output. Need UserCreateRequest (with password) and UserResponse (without).

Q: What is layered architecture violation here?

A: CategoryController returns ProductDto from service directly -- controller knows about external API shape.

Q: MapStruct vs manual mapper?

A: MapStruct generates type-safe mapping at compile time. Manual ProductMapper is error-prone and does not scale.

Chapter 12: Interview Preparation

How to Use This Chapter

Each major topic includes 10 beginner, 10 intermediate, 10 advanced, and 10 scenario-based questions with answers. Practice explaining answers aloud. Reference productService code when possible. Be honest about project weaknesses -- interviewers value awareness of limitations.

Backend & Module Overview

Beginner (10 Questions)

Q: What is backend development?
A: Server-side logic handling requests, business rules, and data access. productService is a Spring Boot backend proxying FakeStore.

Q: What is productService?
A: Spring Boot 4.1 app on Java 17 that exposes REST APIs and calls fakestoreapi.com via RestClient.

Q: What is a REST API?
A: HTTP-based API operating on resources identified by URLs, returning JSON.

Q: What is JSON?
A: JavaScript Object Notation; data format used for API request/response bodies.

Q: What port does the app run on?
A: Default 8080 unless changed in application.properties.

Q: What is Maven?
A: Build tool managing dependencies (pom.xml) and compiling the Java project.

Q: What is Lombok?
A: Library reducing boilerplate via @Getter, @Setter annotations on models/DTOs.

Q: What is an API endpoint?
A: A specific URL + HTTP method combination, e.g., GET /products.

Q: What is Postman used for?
A: Testing HTTP APIs manually without writing client code.

Q: What is fakestoreapi.com?
A: Free fake e-commerce REST API used as external data source.

Intermediate (10 Questions)

Q: What is a BFF?
A: Backend-for-Frontend: API layer tailored for client needs. productService acts as BFF over FakeStore.

Q: Why proxy external API?
A: Hide upstream URL, add error handling, transform data, enable future backend swap.

Q: What is layered architecture?
A: Separating concerns into layers: controller, service, data access. Here: controller, service, RestClient.

Q: What is constructor injection?
A: Dependencies provided via constructor parameters; Spring auto-wires them.

Q: What is application.properties?

A: Externalized config file; stores fakestore.api.base-url.

Q: What is the difference between model and DTO?

A: DTO matches API JSON; model is internal domain. ProductDto.image vs Product.imageUrl.

Q: What is Actuator?

A: Spring Boot health/metrics endpoints; dependency present but not configured.

Q: What is DevTools?

A: Development dependency enabling hot reload; should not be in production.

Q: What packages exist in productService?

A: controllers, services, dtos, models, mappers, config, exceptions.

Q: What is wrong with Main2.java?

A: Dead scratch code in main package; should be deleted.

Advanced (10 Questions)

Q: How would you add a database?

A: New service implementation using JPA repository; swap via @Profile or @Qualifier.

Q: How to make app cloud-ready?

A: Externalize config, health probes, stateless design, containerize with Docker.

Q: What is 12-factor app compliance?

A: Config in env, stateless processes, port binding. Partially met.

Q: How to handle FakeStore downtime?

A: Circuit breaker, fallback cache, meaningful 503 responses.

Q: What is API versioning strategy?

A: URL prefix /v1, header versioning, or content negotiation.

Q: How to structure microservices from this?

A: Split product, cart, user into separate deployable services with API gateway.

Q: What observability is missing?

A: Structured logging, metrics (Micrometer), distributed tracing.

Q: How to secure the service?

A: Spring Security, JWT, HTTPS, input sanitization.

Q: What is technical debt in this project?

A: No tests, no validation, password leaks, inconsistent DTOs, dead code.

Q: How to document APIs?

A: springdoc-openapi generating Swagger UI from controller annotations.

Scenario-Based (10 Questions)

Q: Client reports slow GET /products. Diagnose?

A: Check FakeStore latency, add caching (@Cacheable), add RestClient timeout, consider pagination.

Q: Need to swap FakeStore for real DB. Steps?

A: Create DatabaseProductService implementing ProductService, add JPA entities, switch @Primary bean.

Q: Product team wants rate field in response. Fix?

A: Add rating to Product model, update ProductMapper to map RatingDto, return in response DTO.

Q: Deploy to AWS. Architecture?

A: ECS/EKS container, ALB with TLS, env vars for base URL, CloudWatch logs.

Q: Two developers conflict on ProductController. Resolve?

A: Git merge/rebase, code review, ensure tests pass, separate concerns into different PRs.

Q: Frontend needs only id, title, price. Optimize?

A: Create ProductSummaryDto, new endpoint or projection, avoid returning full Product.

Q: FakeStore changes image to imageUrl. Impact?

A: Update ProductDto field or @JsonProperty, update mapper, regression test.

Q: Need audit trail for product changes.

A: Add createdAt/lastUpdatedAt to BaseModel, populate in service, or use JPA @EntityListeners.

Q: 1000 concurrent users hit /products.

A: Add caching, connection pooling, consider CDN for static product data, horizontal scaling.

Q: Compliance requires no external data transfer.

A: Replace FakeStore with local database, remove RestClient dependency.

Git & Version Control

Beginner (10 Questions)

Q: What is Git?

A: Distributed version control tracking code changes as commits.

Q: What is a commit?

A: Snapshot of code at a point in time with a descriptive message.

Q: What is a branch?

A: Independent line of development, e.g., feature/cart-api.

Q: What is git clone?

A: Copy a remote repository to your local machine.

Q: What is git status?

A: Shows modified, staged, and untracked files.

Q: What is git add?

A: Stage changes for the next commit.

Q: What is git push?

A: Upload local commits to remote repository.

Q: What is git pull?

A: Fetch and merge remote changes into current branch.

Q: What is a remote?

A: Remote repository reference, typically named origin.

Q: What is .gitignore?

A: File listing paths Git should not track, e.g., target/.

Intermediate (10 Questions)

Q: Merge vs rebase?

A: Merge combines histories with merge commit. Rebase replays commits for linear history.

Q: What is git stash?

A: Temporarily shelve uncommitted changes.

Q: What is cherry-pick?

A: Apply a specific commit from one branch to another.

Q: What is a fork?

A: Your copy of someone else's GitHub repository.

Q: What is a Pull Request?

A: Proposal to merge changes with code review and CI checks.

Q: What is a merge conflict?

A: Same lines changed differently in two branches; requires manual resolution.

Q: What is git log?

A: View commit history.

Q: What is git diff?

A: Show changes between commits or working directory.

Q: What is git reset?

A: Move branch pointer; can undo commits (dangerous on shared branches).

Q: What is git revert?

A: Create new commit undoing a previous one; safe for shared branches.

Advanced (10 Questions)

Q: When to rebase vs merge?

A: Rebase feature branches before PR for clean history. Merge to main after review.

Q: What is interactive rebase?

A: git rebase -i to squash, reorder, or edit commits.

Q: What is git bisect?

A: Binary search through commits to find which introduced a bug.

Q: What is a protected branch?

A: Branch requiring PR reviews and passing CI before merge.

Q: What is signed commit?

A: GPG-signed commit verifying author identity.

Q: What is git submodule?

A: Embedded repository within a repository.

Q: What is trunk-based development?

A: Short-lived branches, frequent merges to main.

Q: How to undo last push?

A: git revert on shared branches; never force push main.

Q: What is CODEOWNERS?

A: GitHub file specifying who must review changes to specific paths.

Q: How to structure commits for productService?

A: One logical change per commit: feat, fix, refactor prefixes.

Scenario-Based (10 Questions)

Q: Accidentally committed Main2.java. Fix?
A: git rm Main2.java, commit deletion, add to .gitignore if needed.

Q: Feature branch 20 commits behind main.
A: git fetch origin && git rebase origin/main, resolve conflicts.

Q: Need hotfix on production while feature WIP.
A: git stash, create hotfix branch from main, fix, deploy, git stash pop.

Q: Cherry-pick cart fix to release branch.
A: git cherry-pick <commit-sha> on release branch.

Q: Teammate force-pushed shared branch.
A: Communicate, git fetch, reset carefully, never force push shared branches.

Q: Large binary committed by mistake.
A: git filter-branch or BFG Repo Cleaner to remove from history.

Q: PR has failing CI.
A: Fix locally, push to same branch, CI re-runs automatically.

Q: Split large PR into smaller ones.
A: Create branches per feature area: products, carts, users separately.

Q: Review ProductController changes.
A: Check diff, test endpoints, verify no unrelated changes included.

Q: Release tagging strategy.
A: git tag -a v1.0.0 -m 'Initial release' on main after merge.

Spring Boot, DI & IoC

Beginner (10 Questions)

Q: What is Spring Boot?
A: Framework simplifying Java backend with auto-config and embedded server.

Q: What is @SpringBootApplication?
A: Combines @Configuration, @EnableAutoConfiguration, @ComponentScan.

Q: What is a bean?
A: Object managed by Spring IoC container.

Q: What is @RestController?
A: Marks class as HTTP endpoint handler returning data directly.

Q: What is @Service?
A: Marks class as business logic component.

Q: What is @Autowired?
A: Injects dependency (constructor injection preferred).

Q: What is @Configuration?
A: Class defining @Bean methods.

Q: What is @Bean?
A: Method producing a bean registered in Spring context.

Q: What is IoC?
A: Framework controls object creation instead of application code.

Q: What is DI?

A: Dependencies provided to a class rather than created internally.

Intermediate (10 Questions)

Q: Constructor vs field injection?

A: Constructor: immutable, testable, explicit. Field: discouraged.

Q: What is component scanning?

A: Spring discovers @Service, @RestController in base package and sub-packages.

Q: What is auto-configuration?

A: Conditional bean setup based on classpath and properties.

Q: What is a starter dependency?

A: Curated dependency bundle, e.g., spring-boot-starter-webmvc.

Q: Default bean scope?

A: Singleton: one instance per application context.

Q: What is @Value?

A: Injects property value from application.properties.

Q: How does Spring pick ProductService impl?

A: Only one @Service implements interface; auto-wired by type.

Q: What is @Primary?

A: Preferred bean when multiple implementations exist.

Q: What is @Qualifier?

A: Specifies which bean to inject by name.

Q: What is bean lifecycle?

A: Instantiate, inject, @PostConstruct, use, @PreDestroy.

Advanced (10 Questions)

Q: How does @Conditional work?

A: Register beans only when conditions met (classpath, properties).

Q: What is ApplicationContext?

A: Spring IoC container holding all beans and their dependencies.

Q: Circular dependency problem?

A: A needs B, B needs A. Constructor injection fails; use @Lazy or redesign.

Q: What is @Profile?

A: Activate beans per environment: dev, staging, prod.

Q: How to test with @MockBean?

A: @WebMvcTest replaces real service with mock for controller tests.

Q: What is ConfigurationProperties?

A: Type-safe binding of properties to a class vs scattered @Value.

Q: BeanFactoryPostProcessor role?

A: Modify bean definitions before instantiation.

Q: How RestClient bean is created?

A: RestClientConfig @Configuration with @Bean method using builder pattern.

Q: Why is ProductMapper not a bean?
A: Static utility class; not annotated or registered.

Q: Spring Boot 4.1 changes?
A: Latest Spring Framework 7.x, RestClient as default sync client.

Scenario-Based (10 Questions)

Q: Two ProductService implementations needed.
A: Add @Primary on FakeStore impl, or @Qualifier in controller.

Q: RestClient needs auth header for new API.
A: Register ClientHttpRequestInterceptor on RestClient builder.

Q: Switch FakeStore URL per environment.
A: @Profile dev/prod with different @Value or ConfigurationProperties.

Q: Controller test without hitting FakeStore.
A: @WebMvcTest + @MockBean ProductService, MockMvc perform GET.

Q: App fails: NoUniqueBeanDefinitionException.
A: Multiple beans of same type; add @Primary or @Qualifier.

Q: Need prototype scope for request builder.
A: @Scope(ConfigurableBeanFactory.SCOPE_PROTOTYPE) on helper bean.

Q: Slow startup with many beans.
A: Use spring-context-indexer, lazy initialization, exclude unused auto-config.

Q: Migrate from @Autowired field to constructor.
A: Add final field + constructor; remove @Autowired annotation.

Q: Custom health indicator for FakeStore.
A: Implement HealthIndicator checking RestClient GET /products.

Q: Graceful shutdown during RestClient call.
A: Configure server.shutdown=graceful, timeout, handle interruption.

REST, HTTP & HTTPS

Beginner (10 Questions)

Q: What is HTTP?
A: Protocol for web communication between client and server.

Q: What is GET?
A: Read resource; safe and idempotent.

Q: What is POST?
A: Create resource; not idempotent.

Q: What is PUT?
A: Update/replace resource; idempotent.

Q: What is DELETE?
A: Remove resource; idempotent.

Q: What is status 200?
A: OK; request succeeded.

Q: What is status 404?

A: Not Found; resource does not exist.

Q: What is status 500?

A: Internal Server Error.

Q: What is JSON Content-Type?

A: application/json header indicating JSON body.

Q: What is HTTPS?

A: HTTP encrypted with TLS/SSL.

Intermediate (10 Questions)

Q: Idempotent methods?

A: GET, PUT, DELETE: same result if repeated. POST is not.

Q: When 201 vs 200?

A: 201 for resource creation (POST). 200 for reads and updates.

Q: When 204?

A: Success with no body, e.g., DELETE.

Q: What is @PathVariable?

A: Binds URL path segment to method parameter.

Q: What is @RequestParam?

A: Binds query string parameter.

Q: What is @RequestBody?

A: Deserializes request body JSON to Java object.

Q: What is ResponseEntity?

A: Wrapper for status code, headers, and body.

Q: REST naming conventions?

A: Use nouns (/products), not verbs (/getProducts).

Q: What is TLS handshake?

A: Client-server negotiation establishing encrypted connection.

Q: Query param vs path variable?

A: Path identifies resource (/products/5). Query filters/options (?limit=5).

Advanced (10 Questions)

Q: HATEOAS?

A: Hypermedia links in responses. Not used in productService.

Q: Content negotiation?

A: Accept header determines response format (JSON, XML).

Q: CORS?

A: Cross-Origin Resource Sharing; needed if frontend on different domain.

Q: HTTP/2 benefits?

A: Multiplexing, header compression. Tomcat supports in Spring Boot.

Q: Rate limiting approaches?

A: API gateway, bucket4j, Spring Cloud Gateway filters.

Q: API idempotency keys?

A: Client sends Idempotency-Key header to prevent duplicate POST processing.

Q: RFC 7807 Problem Details?

A: Standard error format with type, title, status, detail fields.

Q: PATCH vs PUT?

A: PATCH partial update; PUT full replacement.

Q: Conditional requests?

A: ETag/If-None-Match for caching; 304 Not Modified.

Q: HTTP method override?

A: X-HTTP-Method-Override for clients that cannot send PUT/DELETE.

Scenario-Based (10 Questions)

Q: Client sends both limit and sort.

A: Current code ignores sort. Fix: combine params or return 400.

Q: DELETE returns 200 with body.

A: Wrong; should return 204 No Content as ProductController does.

Q: Password in GET /users response.

A: Security flaw; create response DTO without password field.

Q: Invalid JSON in POST body.

A: Add @Valid; handle HttpMessageNotReadableException -> 400.

Q: Need HTTPS in development.

A: Configure server.ssl in application.properties with keystore.

Q: Frontend on localhost:3000, API on 8080.

A: Enable CORS via @CrossOrigin or WebMvcConfigurer.

Q: API versioning without breaking clients.

A: Add /api/v1 prefix, keep v0 deprecated for transition period.

Q: Large product list times out.

A: Add pagination, compression (gzip), caching.

Q: Auth login returns 502.

A: FakeStore auth may not exist; implement local auth or handle gracefully.

Q: Client needs request tracing.

A: Add X-Request-Id header, propagate through logs and responses.

RestClient & External APIs

Beginner (10 Questions)

Q: What is RestClient?

A: Spring synchronous HTTP client for calling REST APIs.

Q: Where is RestClient configured?

A: RestClientConfig.java with @Bean and baseUrl.

Q: What is base URL?

A: https://fakestoreapi.com configured in application.properties.

Q: What does .get() do?
A: Starts building an HTTP GET request.

Q: What does .uri() do?
A: Sets request path, e.g., /products/5.

Q: What does .retrieve() do?
A: Executes request and prepares response handling.

Q: What does .body() do?
A: Deserializes response JSON to Java class.

Q: What is ProductDto?
A: Java class matching FakeStore product JSON structure.

Q: What is RestTemplate?
A: Legacy synchronous HTTP client, replaced by RestClient.

Q: What is WebClient?
A: Reactive non-blocking HTTP client for WebFlux.

Intermediate (10 Questions)

Q: RestClient vs RestTemplate?
A: RestClient is modern fluent API; RestTemplate in maintenance mode.

Q: RestClient vs WebClient?
A: RestClient is blocking/sync. WebClient is async/reactive.

Q: What is retrieve() vs exchange()?
A: retrieve() is simplified; exchange() gives full ResponseEntity control.

Q: What is RestClientResponseException?
A: Thrown on 4xx/5xx responses from retrieve().

Q: How to handle 404?
A: Catch exception, check status, throw domain exception.

Q: What is toBodilessEntity()?
A: For responses without body, e.g., DELETE returning 204.

Q: URI template variables?
A: .uri("/products/{id}", productId) expands {id}.

Q: How is JSON deserialized?
A: Jackson ObjectMapper auto-configured by Spring Boot.

Q: What is .post().body()?
A: Sends POST with request body serialized to JSON.

Q: Shared RestClient bean?
A: Single bean in RestClientConfig used by all services.

Advanced (10 Questions)

Q: Configure timeouts?
A: RestClient.builder().requestFactory with timeout settings.

Q: Add request interceptor?
A: builder.requestInterceptor for logging, auth headers.

Q: Retry failed requests?
A: Resilience4j @Retry or Spring Retry template.

Q: Circuit breaker?
A: Resilience4j @CircuitBreaker wrapping RestClient calls.

Q: Test without real API?
A: MockRestServiceServer or WireMock stubbing responses.

Q: Connection pooling?
A: Configure HttpClient 5 connection manager on request factory.

Q: Multiple base URLs?
A: Separate RestClient @Bean per external service.

Q: Error response body parsing?
A: ex.getResponseBodyAsString() on RestClientResponseException.

Q: onStatus() handler?
A: retrieve().onStatus(status -> true, handler) for custom error logic.

Q: Virtual threads with RestClient?
A: Java 21 virtual threads make blocking RestClient more scalable.

Scenario-Based (10 Questions)

Q: FakeStore returns 500.
A: GlobalExceptionHandler maps RestClientResponseException to 502.

Q: Auth login fails silently.
A: FakeStoreAuthImplementation has no try-catch; add 401 mapping.

Q: Need to log all outbound calls.
A: Add ClientHttpRequestInterceptor logging URI and status.

Q: RestClient hangs forever.
A: No timeout configured; add connect/read timeouts immediately.

Q: Migrate from RestTemplate.
A: Replace RestTemplate calls with RestClient fluent API.

Q: Parallel calls to multiple APIs.
A: Use WebClient or virtual threads with CompletableFuture.

Q: FakeStore rate limits requests.
A: Add retry with backoff, cache responses, respect Retry-After header.

Q: Response does not match DTO.
A: Jackson fails deserialization; add @JsonIgnoreProperties(ignoreUnknown=true).

Q: Need to send custom header.
A: restClient.get().uri(...).header("X-Api-Key", key).retrieve()...

Q: ExternalApiException never thrown.
A: Refactor services to throw it instead of raw RestClientResponseException.

Exception Handling

Beginner (10 Questions)

Q: What is an exception?
A: Error condition disrupting normal program flow.

Q: What is try-catch?
A: Handle exceptions locally in a code block.

Q: What is @ExceptionHandler?
A: Method handling specific exception type globally.

Q: What is @RestControllerAdvice?
A: Global exception handler for all REST controllers.

Q: What is ProductNotFoundException?
A: Custom exception when product ID not found.

Q: What is ErrorResponseDto?
A: Standard JSON error body with message, status, path, timestamp.

Q: What is 404 status?
A: Resource not found HTTP status.

Q: What is RuntimeException?
A: Unchecked exception not requiring throws declaration.

Q: What is GlobalExceptionHandler?
A: Centralized exception-to-HTTP mapping class.

Q: Why custom exceptions?
A: Domain-specific errors with meaningful messages.

Intermediate (10 Questions)

Q: Where to catch RestClient 404?
A: Service layer, throw domain exception, handler maps to HTTP.

Q: ProductNotFoundException vs ResourceNotFoundException?
A: Product-specific vs generic; both return 404.

Q: What does handleGeneric catch?
A: All unhandled Exception -> 500 Internal Server Error.

Q: What is ExternalApiException?
A: Custom exception for upstream failures; defined but never thrown.

Q: Exception handler priority?
A: Most specific exception type matched first.

Q: What is WebRequest?
A: Provides request metadata like URI path for error response.

Q: Should controllers have try-catch?
A: No; let exceptions propagate to GlobalExceptionHandler.

Q: What is @ResponseStatus?
A: Alternative to handler method; sets HTTP status on exception class.

Q: Logging in exception handlers?
A: Should log exception details; currently missing in project.

Q: Testing exception handlers?
A: MockMvc perform request, assert status and JSON error body.

Advanced (10 Questions)

Q: RFC 7807 ProblemDetail?
A: Spring 6+ standard error response; replaces custom ErrorResponseDto.

Q: Exception handler order?
A: @Order annotation when multiple advice classes exist.

Q: ControllerAdvice base packages?
A: @RestControllerAdvice(basePackages=...) to limit scope.

Q: Exception as control flow?
A: Acceptable in Spring MVC for not-found; debatable for business logic.

Q: Global vs local handling?
A: Global for consistent API errors; local only for recoverable cases.

Q: Sensitive data in error responses?
A: Never expose stack traces; log server-side only.

Q: Validation exception handling?
A: MethodArgumentNotValidException -> 400 with field errors.

Q: Async exception handling?
A: AsyncUncaughtExceptionHandler for @Async methods.

Q: Filter-level exceptions?
A: Filter exceptions bypass DispatcherServlet; need separate handling.

Q: Correlation ID in errors?
A: Include request ID in ErrorResponseDto for support debugging.

Scenario-Based (10 Questions)

Q: GET /products/999 returns 500 not 404.
A: Service not catching 404; verify ProductNotFoundException thrown.

Q: All errors return 'Internal server error'.
A: handleGeneric too broad; add specific handlers, log exceptions.

Q: Client wants field-level validation errors.
A: Add @Valid, handle MethodArgumentNotValidException with field map.

Q: Upstream 429 rate limit.
A: Catch RestClientResponseException, map to 429 or 503 with Retry-After.

Q: Auth failure returns 502.
A: Add handler for 401 from auth endpoint specifically.

Q: Error path exposes internal URI.
A: Customize path in ErrorResponseDto; sanitize sensitive info.

Q: Two advice classes conflict.
A: Use @Order; ensure no overlapping @ExceptionHandler signatures.

Q: Need different errors for dev vs prod.
A: @Profile on advice class or conditional error detail.

Q: NullPointerException in mapper.
A: Caught by handleGeneric as 500; fix null check in ProductMapper.

Q: Integration test for 404.
A: MockRestServiceServer return 404, assert ErrorResponseDto JSON structure.

productService Architecture & APIs

Beginner (10 Questions)

Q: How many controllers?
A: Five: Product, Category, Cart, User, Auth.

Q: What does ProductController do?
A: CRUD operations on /products endpoints.

Q: What is ProductMapper?
A: Converts ProductDto to Product domain model.

Q: What is FakeStoreProductImplementation?
A: @Service implementing ProductService via RestClient.

Q: What is a service interface?
A: Contract like ProductService defining available operations.

Q: What is RestClientConfig?
A: Creates RestClient bean with FakeStore base URL.

Q: What is GET /categories?
A: Returns list of category names from FakeStore.

Q: What is POST /auth/login?
A: Sends credentials, returns token from FakeStore.

Q: What is CartDto?
A: Data object for cart with id, userId, date, products list.

Q: What is the project Java version?
A: Java 17 as specified in pom.xml.

Intermediate (10 Questions)

Q: Why service interface + implementation?
A: Decoupling, testability, ability to swap data source.

Q: Inconsistent return types?
A: ProductController returns Product; CategoryController returns ProductDto.

Q: Category name vs ID fix?
A: Endpoints use String categoryName matching FakeStore API.

Q: What endpoints lack 404 handling?
A: List endpoints and category lookups.

Q: Password security issue?
A: UserDto.password returned in GET responses.

Q: What is missing validation?
A: No @Valid on any @RequestBody parameter.

Q: RatingDto handling?
A: Present in ProductDto but ignored by ProductMapper.

Q: BaseModel unused fields?
A: createdAt, lastUpdatedAt, isDeleted never populated.

Q: Auth implementation quality?
A: No error handling; token not validated on requests.

Q: How many total endpoints?
A: 20 across all controllers.

Advanced (10 Questions)

Q: Hexagonal architecture migration?
A: Define ports (inbound/outbound), adapters for controllers and FakeStore.

Q: CQRS for productService?
A: Separate read/write models and endpoints for complex queries.

Q: Event-driven extension?
A: Publish ProductCreated event after POST for downstream systems.

Q: Multi-tenant support?
A: Tenant ID in header, filter data per tenant in service layer.

Q: Saga pattern for cart checkout?
A: Coordinate cart, inventory, payment across services.

Q: API composition?
A: Single endpoint aggregating product + category + rating from multiple calls.

Q: GraphQL vs REST?
A: GraphQL for flexible queries; REST simpler for CRUD proxy.

Q: Contract testing?
A: Pact tests verifying productService matches FakeStore contract.

Q: Feature flags?
A: Toggle new endpoints without deployment using LaunchDarkly or similar.

Q: Dead code audit?
A: Main2.java, unused ExternalApiException, commented BaseModel annotations.

Scenario-Based (10 Questions)

Q: Add product search by title.
A: New endpoint GET /products/search?q=, pass to FakeStore or filter locally.

Q: Cart total calculation.
A: Service method summing CartItemDto price * quantity before returning.

Q: Hide deleted products.
A: Filter isDeleted in service if BaseModel populated from future DB.

Q: Add product image upload.
A: New endpoint with MultipartFile; store in S3, save URL in product.

Q: Implement wishlist feature.
A: New controller, service, DTO; store in database not FakeStore.

Q: CategoryController returns Product not ProductDto.
A: Align with ProductController or standardize on response DTO.

Q: Load test reveals thread exhaustion.
A: Add RestClient timeouts, consider WebClient or virtual threads.

Q: Penetration test finds password leak.
A: Emergency fix: @JsonIgnore on password, deploy immediately.

Q: ProductService needs offline mode.
A: Cache last-known products, serve from cache when FakeStore down.

Q: Monolith to microservices split.
A: Extract cart-service, user-service with independent deployments.

Advanced Topics

Beginner (10 Questions)

Q: What is SOLID?
A: Five OOP design principles for maintainable code.

Q: What is S in SOLID?
A: Single Responsibility: one reason to change per class.

Q: What is D in SOLID?
A: Dependency Inversion: depend on abstractions not concretions.

Q: What is validation?
A: Checking input data meets rules before processing.

Q: What is @NotNull?
A: Bean Validation constraint: field must not be null.

Q: What is pagination?
A: Returning data in pages instead of entire dataset.

Q: What is caching?
A: Storing frequently accessed data for faster retrieval.

Q: What is @Cacheable?
A: Spring annotation caching method return value.

Q: What is MapStruct?
A: Compile-time code generator for bean mapping.

Q: What is layered architecture?
A: Organizing code into presentation, business, data layers.

Intermediate (10 Questions)

Q: DTO vs Entity when to use?
A: DTO at API boundary; entity for domain/persistence logic.

Q: SOLID in productService?
A: DI demonstrates D; handler violates S (too many concerns).

Q: Add @Valid workflow?
A: Annotate DTO fields, add @Valid on parameter, handle validation exception.

Q: Caching strategy for products?
A: @Cacheable on getAllProducts, TTL 5 min, evict on POST/PUT/DELETE.

Q: Pagination with FakeStore?
A: Pass limit and offset as query params; wrap in Page response.

Q: MapStruct vs ProductMapper?
A: MapStruct generates compile-time safe mapping; less boilerplate.

Q: Separate request/response DTOs?
A: UserCreateRequest with password; UserResponse without.

Q: CQRS basics?
A: Separate models for commands (writes) and queries (reads).

Q: Interface segregation example?
A: Separate ProductReadService and ProductWriteService interfaces.

Q: Open/Closed in services?

A: Add new FakeStore impl without modifying controller.

Advanced (10 Questions)

Q: Cache invalidation strategies?

A: TTL, event-driven eviction, write-through, cache-aside.

Q: Distributed caching?

A: Redis cluster shared across multiple app instances.

Q: Validation groups?

A: @Validated(Create.class) vs @Validated(Update.class) for different rules.

Q: Custom validator?

A: Implement ConstraintValidator for cross-field validation.

Q: Domain-driven design?

A: Bounded contexts: product, cart, user as separate domains.

Q: Event sourcing?

A: Store state changes as events; rebuild state from event log.

Q: Read replica pattern?

A: Route reads to replica DB, writes to primary.

Q: API gateway pattern?

A: Kong/AWS API Gateway in front of productService.

Q: Bulkhead pattern?

A: Separate thread pools for product vs cart RestClient calls.

Q: Strangler fig migration?

A: Gradually replace FakeStore calls with database queries.

Scenario-Based (10 Questions)

Q: ProductDto allows negative price.

A: Add @Positive on price field, @Valid on controller parameter.

Q: Cache serves stale product after update.

A: Add @CacheEvict on updateProduct and deleteProduct methods.

Q: MapStruct migration plan.

A: Add dependency, create ProductMapper interface, deprecate static mapper.

Q: Paginate GET /users for 100+ users.

A: Add page/size params, return Page<UserResponseDto> with metadata.

Q: SOLID refactor GlobalExceptionHandler.

A: Split into NotFoundHandler, ValidationHandler, ExternalApiHandler.

Q: Rating missing from API response.

A: Add rating field to Product model, update mapper, add to response DTO.

Q: Validate category name exists.

A: Custom validator checking against getAllCategories() list.

Q: Performance: N+1 on cart items.

A: Not applicable now (no DB); relevant when adding JPA relationships.

Q: Add request rate limiting.
A: Bucket4j filter limiting 100 req/min per IP.

Q: Audit log for admin actions.
A: AOP @Around on DELETE/PUT methods logging user and timestamp.

Quick Revision: Top 20 Cross-Topic Questions

Q: Explain full request flow for GET /products/5.
A: Client -> Tomcat -> DispatcherServlet -> ProductController -> FakeStoreProductImplementation -> RestClient -> FakeStore -> ProductDto -> ProductMapper -> Product -> JSON 200.

Q: What would you fix first in this project?
A: Password leak in UserDto, add @Valid validation, add RestClient timeouts, remove Main2.java.

Q: How does DI work in ProductController?
A: Spring injects ProductService implementation via constructor at startup.

Q: Difference between 404 and 500 in this project?
A: 404: ProductNotFoundException/ResourceNotFoundException. 500: unhandled Exception in handleGeneric.

Q: Why both Product and ProductDto exist?
A: ProductDto matches FakeStore JSON; Product is internal domain model with Category object.

Q: How is category endpoint different from early version?
A: Uses category name string in path, not numeric ID.

Q: What HTTP client and why?
A: RestClient: modern sync client appropriate for simple FakeStore proxy.

Q: How to test without FakeStore?
A: MockRestServiceServer stubbing RestClient responses in @SpringBootTest.

Q: What SOLID principle is violated?
A: Single Responsibility in GlobalExceptionHandler; Open/Closed is followed via service interfaces.

Q: Production deployment checklist?
A: Remove DevTools, add HTTPS, configure Actuator security, add validation, mask passwords, add tests, set timeouts.

Q: Git workflow for adding feature?
A: Branch -> implement -> test -> commit -> push -> PR -> review -> merge.

Q: What is the limit/sort bug?
A: GET /products?limit=5&sort=asc only applies limit; sort ignored.

Q: How does login work?
A: POST /auth/login -> FakeStoreAuthImplementation -> RestClient POST /auth/login -> token returned.

Q: What annotations on ProductController?
A: @RestController, @RequestMapping, @GetMapping, @PostMapping, @PutMapping, @DeleteMapping.

Q: How to add caching?
A: @EnableCaching, @Cacheable on read methods, Redis provider.

Q: What is missing from exception handling?
A: Logging, validation errors (400), auth errors (401), specific upstream error mapping.

Q: How many endpoints and controllers?
A: 20 endpoints, 5 controllers.

Q: What is ExternalApiException status?
A: Dead code: handler exists but no service throws it.

Q: MapStruct benefit over ProductMapper?

A: Compile-time safety, less manual field mapping, supports complex mappings.

Q: Scenario: interviewer asks to design from scratch.

A: Describe layers: controller, service interface, FakeStore impl, RestClient config, DTOs, mapper, exception handler.

Chapter 13: Final Revision Cheat Sheets

Spring Boot Cheat Sheet

Annotation	Purpose	Example
@SpringBootApplication	Main app entry	ProductServiceApplication
@RestController	REST endpoint class	ProductController
@RequestMapping	Base URL path	/products
@GetMapping	HTTP GET handler	getAllProducts()
@PostMapping	HTTP POST handler	addNewProduct()
@PutMapping	HTTP PUT handler	updateProduct()
@DeleteMapping	HTTP DELETE handler	deleteProduct()
@PathVariable	URL path param	productId
@RequestParam	Query param	limit, sort
@RequestBody	Request JSON body	ProductDto
@Service	Business logic bean	FakeStoreProductImpl
@Configuration	Bean config class	RestClientConfig
@Bean	Register bean	restClient()
@Value	Inject property	${fakestore.api.base-url}
@RestControllerAdvice	Global exception handler	GlobalExceptionHandler
@ExceptionHandler	Handle exception type	handleProductNotFound

REST API Cheat Sheet - productService Endpoints

Method	Endpoint	Status	Body
GET	/products	200	List<Product>
GET	/products/{id}	200/404	Product
GET	/products/category/{name}	200	List<Product>
POST	/products	201	ProductDto in
PUT	/products/{id}	200/404	ProductDto in
DELETE	/products/{id}	204/404	none
GET	/categories	200	List<CategoryDto>
GET	/categories/{name}/products	200	List<ProductDto>
GET	/carts	200	List<CartDto>
GET	/carts/{id}	200/404	CartDto
GET	/carts/user/{id}	200	List<CartDto>
POST	/carts	201	CartDto in
PUT	/carts/{id}	200/404	CartDto in
DELETE	/carts/{id}	204/404	none
GET	/users	200	List<UserDto>
GET	/users/{id}	200/404	UserDto

POST	/users	201	UserDto in
PUT	/users/{id}	200/404	UserDto in
DELETE	/users/{id}	204/404	none
POST	/auth/login	200	LoginRequestDto in

Git Cheat Sheet

```
# Daily workflow
git status                    # Check changes
git add <file>                # Stage changes
git commit -m "message"       # Save snapshot
git push origin <branch>      # Upload to remote
git pull origin main          # Get latest main

# Branching
git checkout -b feature/name  # Create + switch branch
git branch                    # List branches
git merge feature/name        # Merge into current
git rebase main               # Rebase onto main

# Utilities
git stash / git stash pop     # Shelve / restore WIP
git cherry-pick <sha>         # Copy one commit
git log --oneline -10         # Recent history
git diff                      # View changes
git revert <sha>              # Safe undo on shared branch
```

HTTP Methods & Rules Cheat Sheet

Method	Purpose	Safe	Idempotent	Body
GET	Read	Yes	Yes	No
POST	Create	No	No	Yes
PUT	Replace	No	Yes	Yes
PATCH	Partial update	No	No	Yes
DELETE	Remove	No	Yes	No

HTTP Status Codes Cheat Sheet

Code	Name	When	productService
200	OK	Success with body	GET, PUT, login
201	Created	Resource created	POST products/carts/users
204	No Content	Success no body	DELETE operations
400	Bad Request	Invalid input	NOT IMPLEMENTED
401	Unauthorized	Auth failed	NOT IMPLEMENTED
404	Not Found	Missing resource	ProductNotFoundException
405	Method Not Allowed	Wrong HTTP verb	Spring default
500	Internal Error	Unhandled exception	handleGeneric
502	Bad Gateway	Upstream failure	RestClientResponseException

HTTP Headers Cheat Sheet

Header	Direction	Purpose	Example
Content-Type	Request/Response	Body format	application/json
Accept	Request	Expected response	application/json
Authorization	Request	Auth token	Bearer <token> (not used)
Location	Response	Created resource URL	Not set on 201
Cache-Control	Both	Caching rules	Not configured

RestClient Cheat Sheet

```
// Bean setup (RestClientConfig.java)
@Bean
public RestClient restClient() {
    return RestClient.builder()
        .baseUrl("https://fakestoreapi.com")
        .build();
}

// GET
ProductDto dto = restClient.get()
    .uri("/products/{id}", id)
    .retrieve()
    .body(ProductDto.class);

// GET with query param
restClient.get().uri("/products?limit={n}", n).retrieve().body(ProductDto[].class);

// POST
ProductDto created = restClient.post()
    .uri("/products").body(dto).retrieve().body(ProductDto.class);

// PUT
restClient.put().uri("/products/{id}", id).body(dto).retrieve().body(ProductDto.class);

// DELETE
restClient.delete().uri("/products/{id}", id).retrieve().toBodilessEntity();

// Error handling
try { ... } catch (RestClientResponseException ex) {
    if (ex.getStatusCode().value() == 404) throw new ProductNotFoundException(id);
}
```

RestClient vs RestTemplate vs WebClient

Feature	RestClient	RestTemplate	WebClient
Style	Fluent builder	Method-per-op	Fluent reactive
Blocking	Yes	Yes	No (async)
Status	Current (Spring 6.1+)	Maintenance	Active (WebFlux)
Used in project	Yes	No	No
Best for	Sync REST calls	Legacy code	High concurrency

Spring Annotations Quick Reference

```
CONTROLLER LAYER          SERVICE LAYER          CONFIG/EXCEPTIONS
@RestController           @Service               @Configuration
@RequestMapping           implements Interface    @Bean
```

```
@GetMapping              @Autowired (ctor)       @Value
@PostMapping                                     @RestControllerAdvice
@PutMapping                                      @ExceptionHandler
@DeleteMapping
@PathVariable
@RequestParam
@RequestBody
```

Exception Handling Cheat Sheet

Exception	HTTP	Handler Method	When
ProductNotFoundException	404	handleProductNotFound	Product ID missing
ResourceNotFoundException	404	handleResourceNotFound	Cart/User missing
RestClientResponseException	varies	handleRestClientError	Upstream error
ExternalApiException	502	handleExternalApi	Never thrown currently
Exception	500	handleGeneric	Any unhandled error

DTO vs Model Field Mapping

ProductDto Field	Product Model Field	Notes
id	id (BaseModel)	Direct map
title	title	Direct map
price	price	Null-safe to 0.0
description	description	Direct map
image	imageUrl	Name mismatch handled
category (String)	category.name	String to object
rating	NOT MAPPED	Data loss bug

Project Structure Cheat Sheet

```
org.pavan.productservice/
|-- ProductServiceApplication.java    (@SpringBootApplication)
|-- controllers/                     (5 REST controllers)
|-- services/                        (5 interfaces + 5 FakeStore impls)
|-- dtos/                            (12 DTO classes)
|-- models/                          (Product, Category, BaseModel)
|-- mappers/                         (ProductMapper - static)
|-- config/                          (RestClientConfig)
|-- exceptions/                      (4 exceptions + handler + ErrorResponseDto)
|-- Main2.java                       (DELETE - dead code)
resources/
|-- application.properties           (fakestore.api.base-url)
```

SOLID Quick Reference

Principle	Meaning	projectService Example
S	One job per class	Controllers only handle HTTP
O	Open for extension	New service impl without chang

L	Substitutable impls	FakeStore impl honors ProductS
I	Small interfaces	Separate CartService, UserServ
D	Depend on abstractions	ProductController -> ProductSe

Known Issues Checklist

Issue	Severity	Fix
Password in UserDto responses	Critical	@JsonIgnore or separate DTO
No input validation	High	@Valid on all POST/PUT
No RestClient timeout	High	Configure request factory
limit+sort bug	Medium	Support both or return 400
RatingDto not mapped	Medium	Update ProductMapper
Main2.java dead code	Low	Delete file
ExternalApiException unused	Low	Use or remove
Mixed Model/DTO returns	Medium	Standardize response DTOs
No tests	High	MockMvc + MockRestServiceServe
Auth token not enforced	Critical	Spring Security filter

Maven Commands Cheat Sheet

```
mvn clean install          # Build + test
mvn spring-boot:run        # Run application
mvn test                   # Run tests only
mvn dependency:tree        # View dependency tree
mvn clean package -DskipTests  # Build without tests
```

Final Revision: Key Numbers

Item	Value
Java version	17
Spring Boot version	4.1.0
Default port	8080
External API	https://fakestoreapi.com
Controllers	5
Total endpoints	20
Service interfaces	5
Service implementations	5
DTO classes	12
Model classes	3
Exception classes	4 custom + 1 handler

Exam Tip

Be able to trace GET /products/5 from HTTP request through DispatcherServlet, ProductController, FakeStoreProductImplementation, RestClient, ProductMapper, and back. Know which exceptions map to which status codes. Acknowledge project weaknesses (password leak, no validation) and state how you would fix them.